

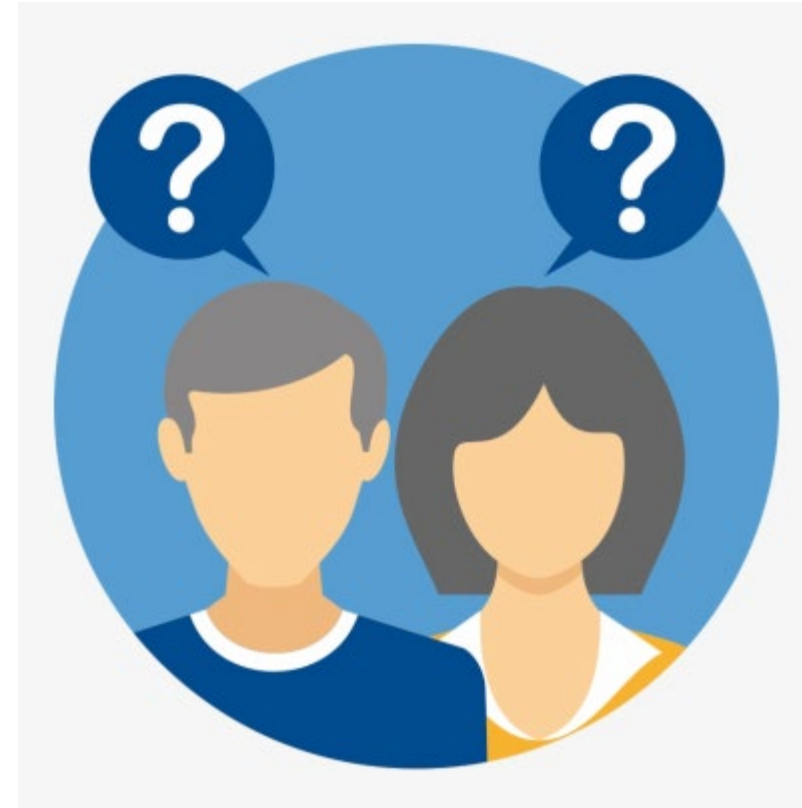


Programación Avanzada

Docente: Javier Diaz Diaz

¿QUIENES SOMOS?

- Me llamo
- Me gusta....
- Soy de ...



[Esta foto](#) de Autor desconocido está bajo licencia [CC BY-NC-ND](#)

JAVIER Díaz Díaz



**Magister Gestión de la Tecnología
Educativa
Ingeniero de Sistemas**

Profesional en Ingeniería de sistemas, Magister en Gestión de la Tecnología Educativa, Diplomados en Docencia universitaria, Certificaciones internacionales en Itil, ICDL, Microsoft y Oracle, Auditor norma ISO 27001:2013

Cuento con experiencia certificada de más de 20 años como docente tanto presencial como virtual en universidades y el SENA en áreas de Tecnología, Diseño y desarrollo de Software, Bases de Datos y Tecnologías de la información y la comunicación.

Con capacidades y habilidades para diseñar, dirigir y evaluar propuestas pedagógicas relacionadas con las tecnologías de la información y la comunicación en ambientes virtuales de aprendizaje.

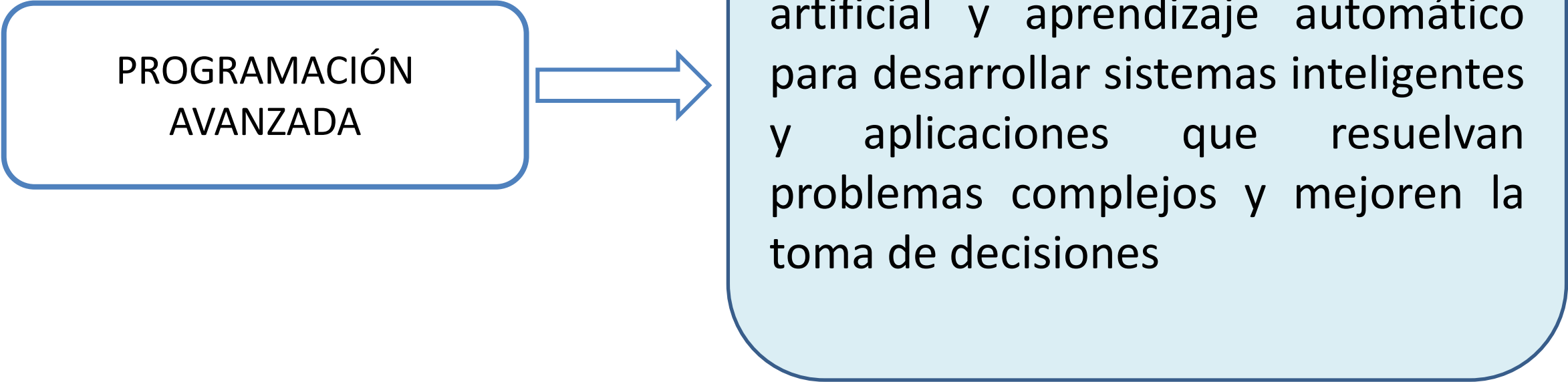
Docente programas de pregrado: UDI, UMB

Docente programas de posgrado: UDES

Formador estrategia Misión Tic 2022 : UIS, UNAB

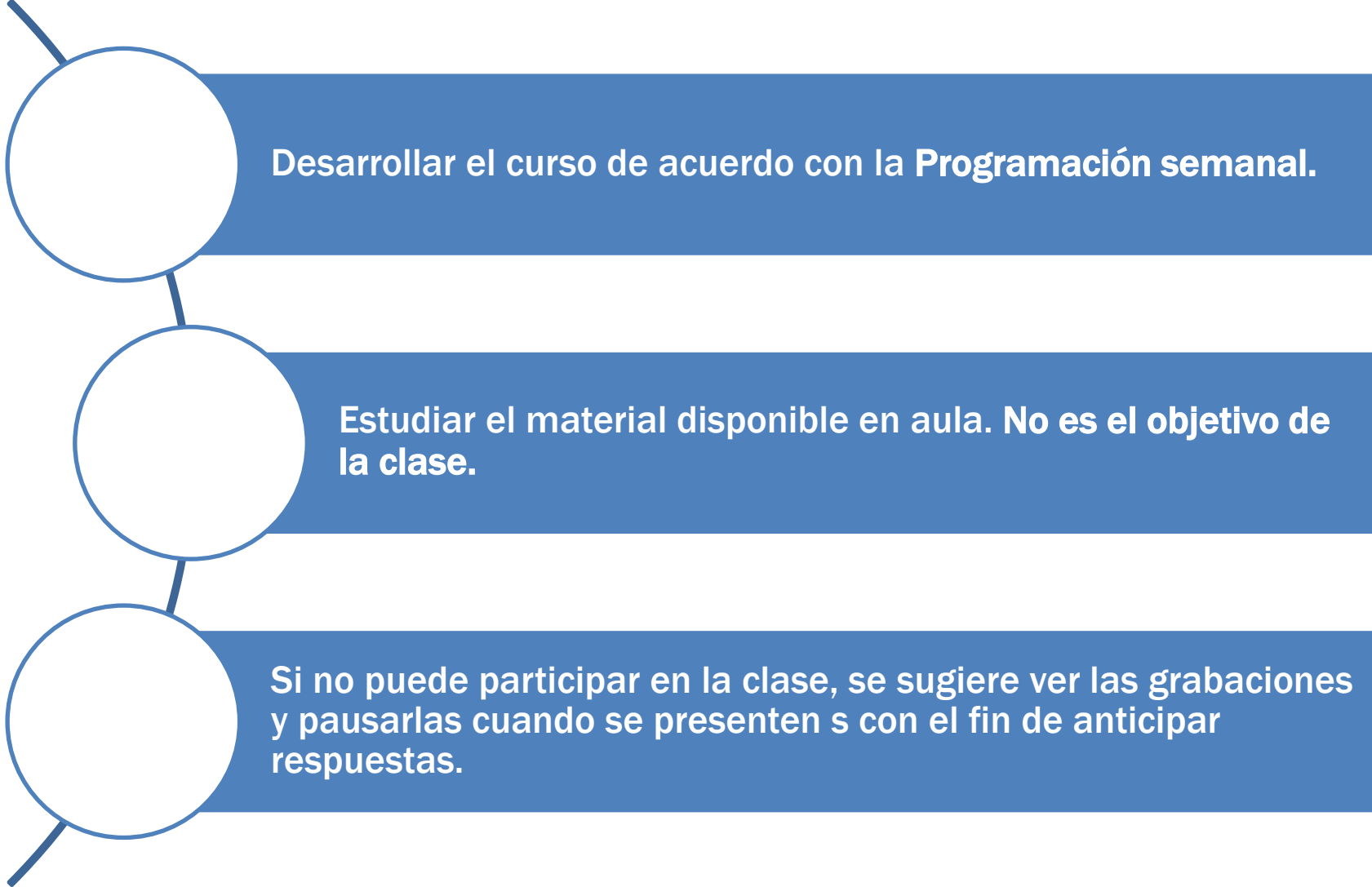
Resultados de Aprendizaje

PROGRAMACIÓN
AVANZADA



Diseñar algoritmos y aplicar técnicas de lógica computacional, inteligencia artificial y aprendizaje automático para desarrollar sistemas inteligentes y aplicaciones que resuelvan problemas complejos y mejoren la toma de decisiones

Recomendaciones Generales

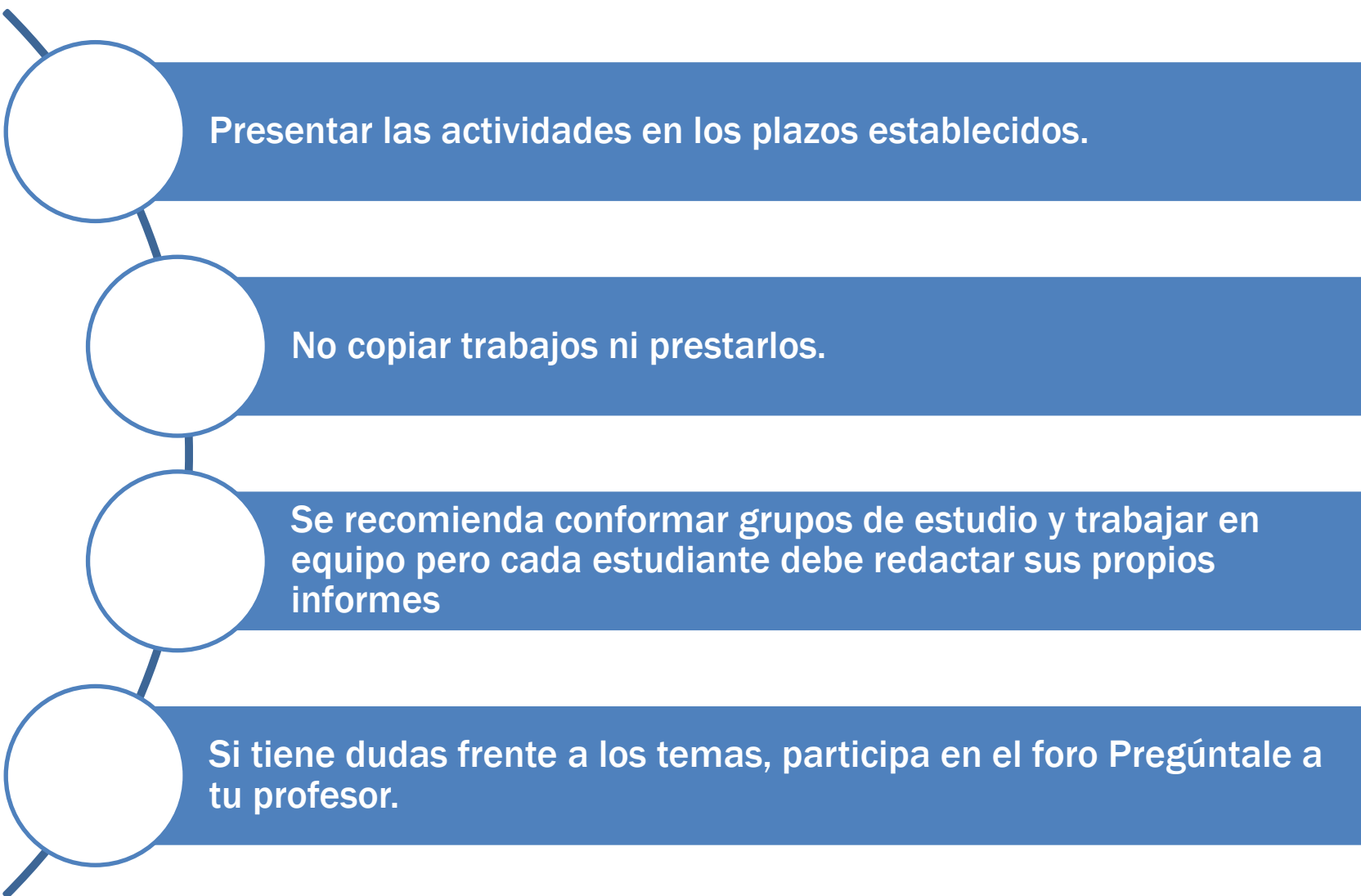


Desarrollar el curso de acuerdo con la **Programación semanal**.

Estudiar el material disponible en aula. **No es el objetivo de la clase.**

Si no puede participar en la clase, se sugiere ver las grabaciones y pausarlas cuando se presenten s con el fin de anticipar respuestas.

Recomendaciones Generales



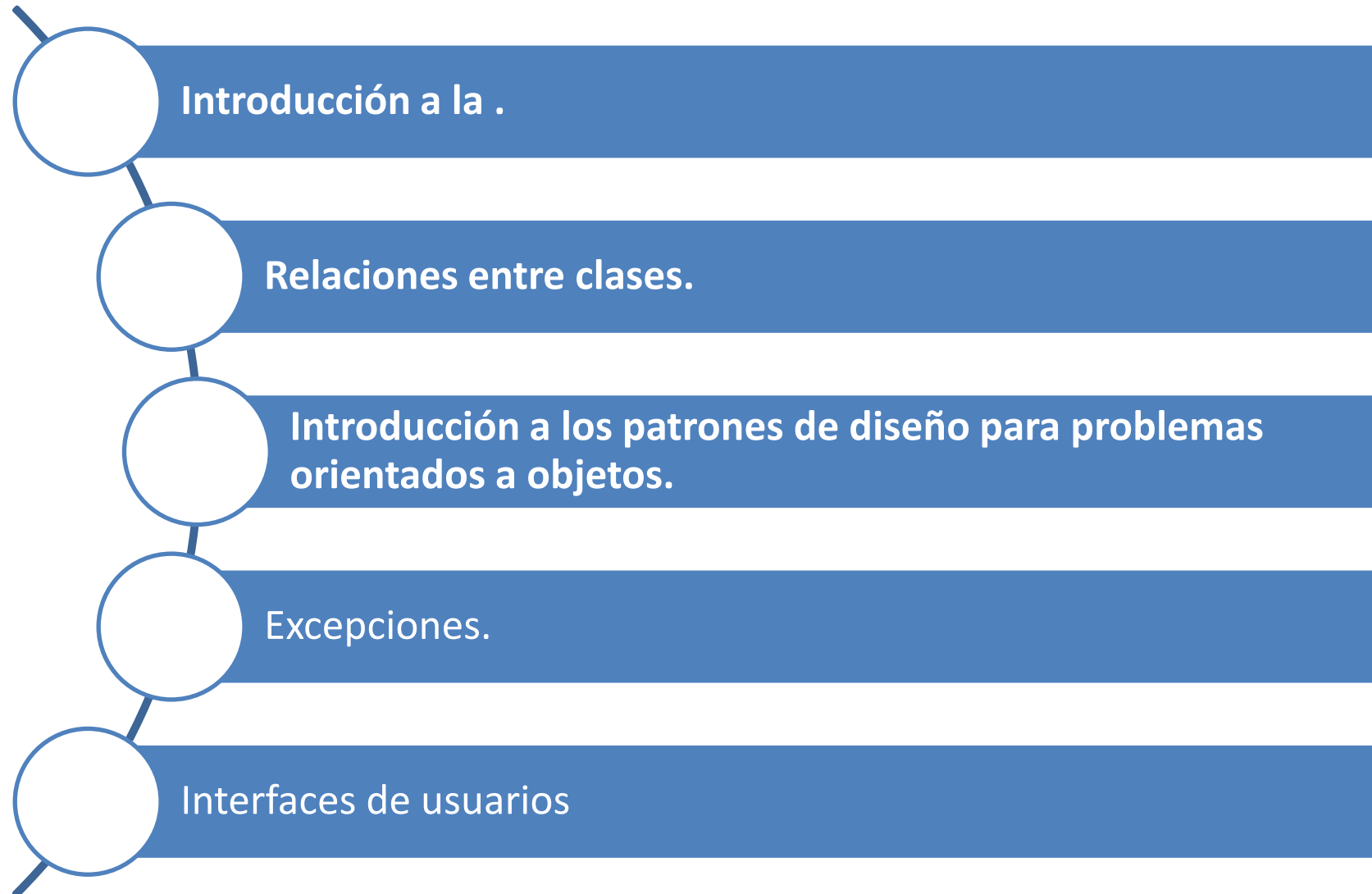
Presentar las actividades en los plazos establecidos.

No copiar trabajos ni prestarlos.

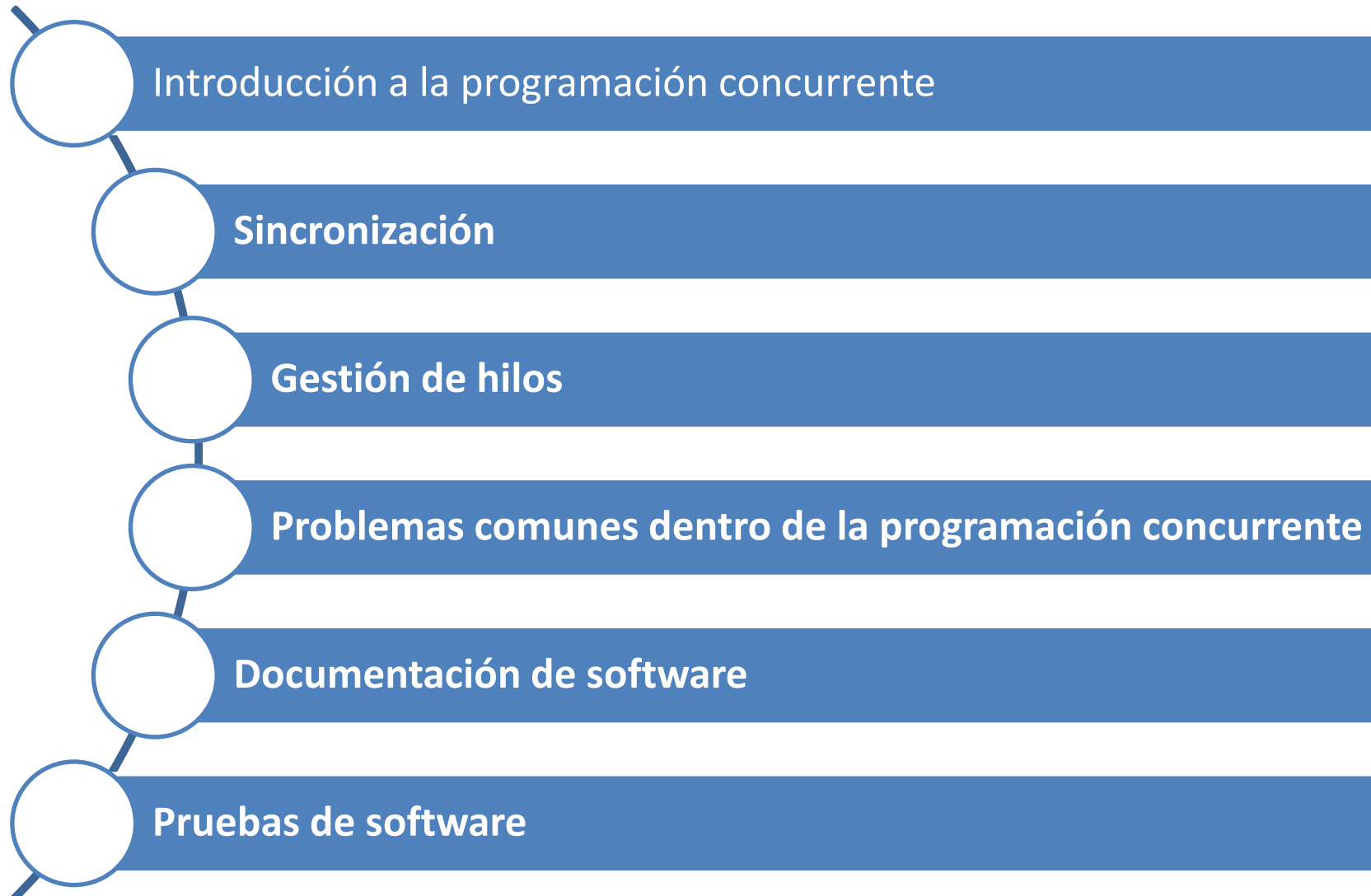
Se recomienda conformar grupos de estudio y trabajar en equipo pero cada estudiante debe redactar sus propios informes

Si tiene dudas frente a los temas, participa en el foro Pregúntale a tu profesor.

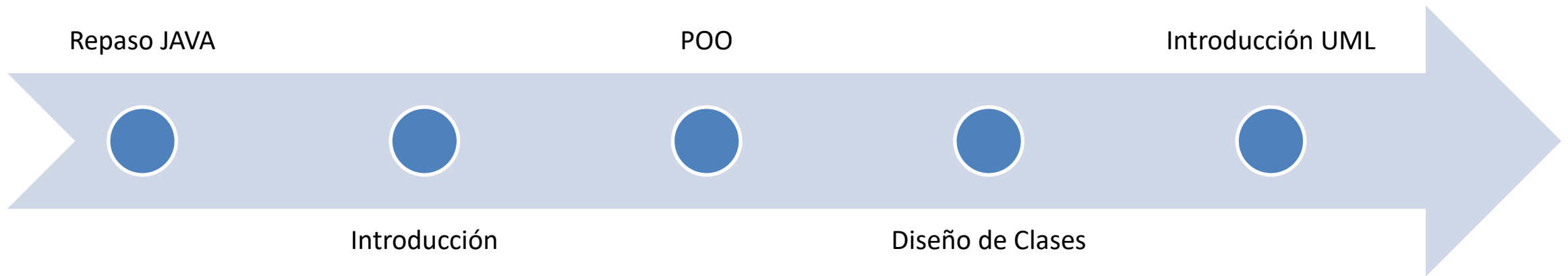
Contenidos



Contenidos



Agenda



REPASO JAVA

Instalación JDK - IDE

Instalación JDK (Java Development Kit)



Instalación IDE (NetBeans)

https://www.youtube.com/watch?v=Stx3MNV3AHE&list=PLCTD_CpMeEKT-qEHGqZH3fkBgXH4GOTF&index=2



Sintaxis – IDE NetBeans (Primer programa)

ProgramaHolaMundo - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

<default config>

Projects Files Services

- ProgramaHolaMundo
 - Source Packages
 - programaholamundo
 - ProgramaHolaMundo.java
 - Test Packages
 - Libraries
 - Test Libraries

Start Page x ProgramaHolaMundo.java x

Source History

```
10 * @author SERGIO
11 */
12 import java.util.Scanner;
13
14 public class ProgramaHolaMundo {
15
16     /**
17      * @param args the command line arguments
18      */
19     public static void main(String[] args) {
20         // Crear instancia de Scanner para entrada por consola
21         Scanner entrada=new Scanner(System.in);
22         System.out.println("Hola Mundo");
23
24     }
25
26 }
27
```

programaholamundo.ProgramaHolaMundo > main >

main - Navigator x

Members <empty>

- ProgramaHolaMundo
 - main(String[] args)

Output - ProgramaHolaMundo (run) x

```
run:
Hola Mundo
BUILD SUCCESSFUL (total time: 2 seconds)
```

Estructuras de Control en Java

 Tipos de Datos

 Operadores

 Estructuras de Control



Tipos de Datos en Java

Tipos de Datos Primitivos en Java

¿Qué son los tipos de datos primitivos en Java?

Como ya hemos comentado [Java](#) es un lenguaje de tipado estático. Es decir, se define el tipo de dato de la variable a la hora de definir esta. Es por ello que todas las variables tendrán un tipo de dato asignado.

El lenguaje [Java](#) da de base una serie de tipos de datos primitivos.

- byte
- short
- int
- long
- float
- double
- boolean
- char





Tipos de Datos en Java

byte

Representa un tipo de dato de 8 bits con signo. De tal manera que puede almacenar los valores numéricos de -128 a 127 (ambos inclusive).

short

Representa un tipo de dato de 16 bits con signo. De esta manera almacena valores numéricos de -32.768 a 32.767.

int

Es un tipo de dato de 32 bits con signo para almacenar valores numéricos. Cuyo valor mínimo es -2^{31} y el valor máximo $2^{31}-1$.

long

Es un tipo de dato de 64 bits con signo que almacena valores numéricos entre -2^{63} a $2^{63}-1$.

float

Es un tipo dato para almacenar números en coma flotante con precisión simple de 32 bits.

double

Es un tipo de dato para almacenar números en coma flotante con doble precisión de 64 bits.

boolean

Sirve para definir tipos de datos booleanos. Es decir, aquellos que tienen un valor de true o false. Ocupa 1 bit de información.

char

Es un tipo de datos que representa a un carácter Unicode sencillo de 16 bits.



Tipos de Datos en Java

Tipos_Datos - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Projects Files Services

- JavaApplication1
 - Source Packages
 - Libraries
- Tipos_Datos
 - Source Packages
 - tipos_datos
 - Tipos_Datos.java
 - Test Packages
 - Libraries
 - Test Libraries

Tipos_Datos - Navigator

Members

- Tipos_Datos
 - main(String[] args)

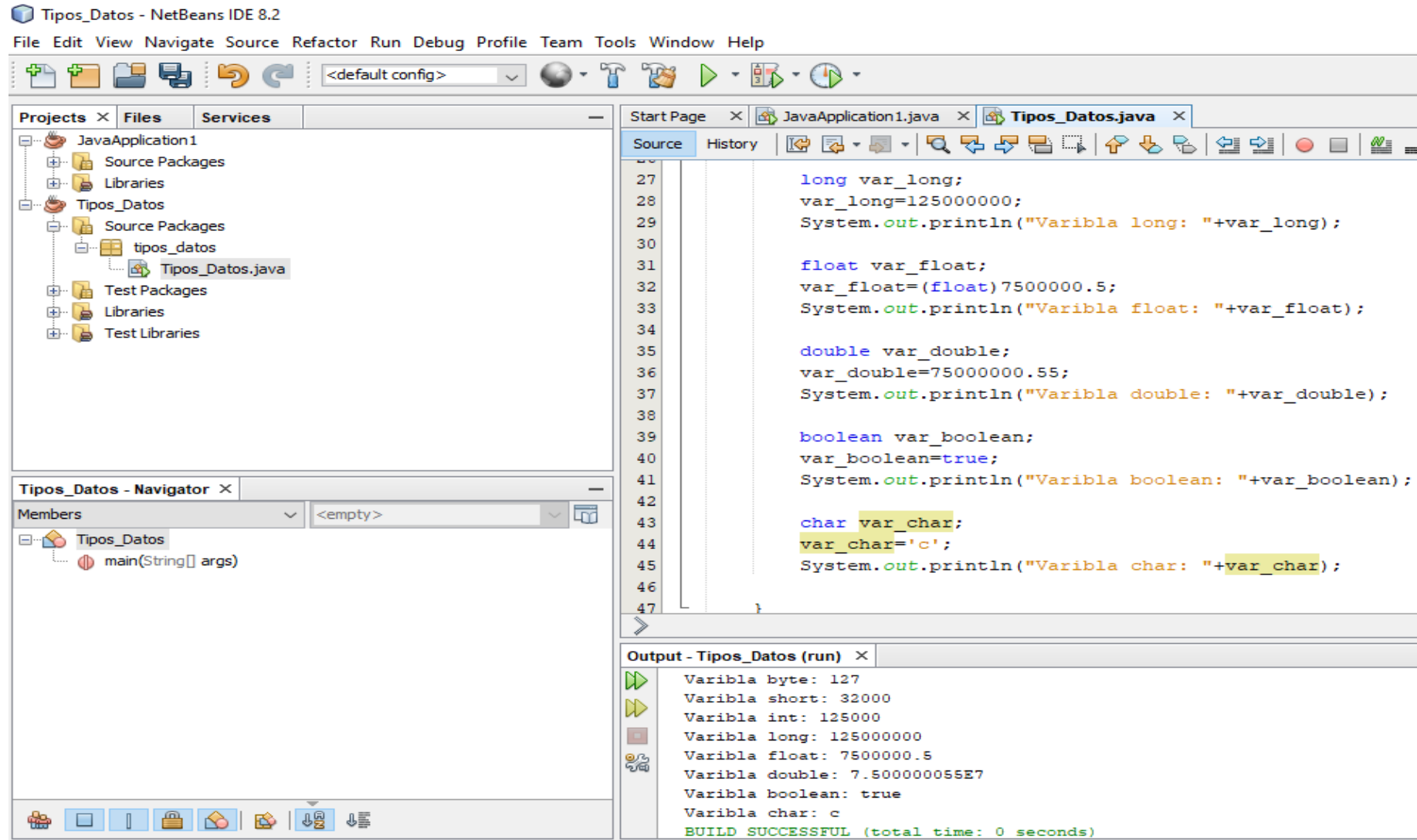
Source

```
6 package tipos_datos;
7
8 /**
9  * @author SERGIO
10 */
11 public class Tipos_Datos {
12
13
14     public static void main(String[] args) {
15         byte var_byte;
16         var_byte=127;
17         System.out.println("Varibla byte: "+var_byte);
18
19         short var_short;
20         var_short=32000;
21         System.out.println("Varibla short: "+var_short);
22
23         int var_int;
24         var_int=125000;
25         System.out.println("Varibla int: "+var_int);
26     }
```

Output - Tipos_Datos (run)

```
Varibla byte: 127
Varibla short: 32000
Varibla int: 125000
Varibla long: 125000000
Varibla float: 7500000.5
Varibla double: 7.500000055E7
Varibla boolean: true
Varibla char: c
BUILD SUCCESSFUL (total time: 0 seconds)
```


Tipos de Datos en Java



Tipos de Datos en Java

Videotutoriales recomendados:

https://www.youtube.com/watch?v=JQEYWxB2gbM&list=PLCTD_CpMeEKTt-qEHGqZH3fkBgXH4GOTF&index=7

https://www.youtube.com/watch?v=zZjNNxaLXdw&list=PLCTD_CpMeEKTt-qEHGqZH3fkBgXH4GOTF&index=8

Operadores Aritméticos

OPERADOR	USO
+	Suma.
-	Resta.
*	Multiplicación.
/	División.
%	Resto de la división o Módulo.
++	Incremento.
--	Decremento.

Operadores Relacionales

Operador.	Uso.
==	Igual que...
!=	Distinto que...
<	Menor que...
<=	Menor o igual que...
>	Mayor que...
>=	Mayor o igual que...

Operadores Lógicos

Operador	Descripción
&&	Operador and (y)
	Operador or (o)
!	Operador not (no)

Estructuras de Control

Condicional - Selectiva

Iterativas - Ciclos



Estructuras de Control Condicional - Selectiva

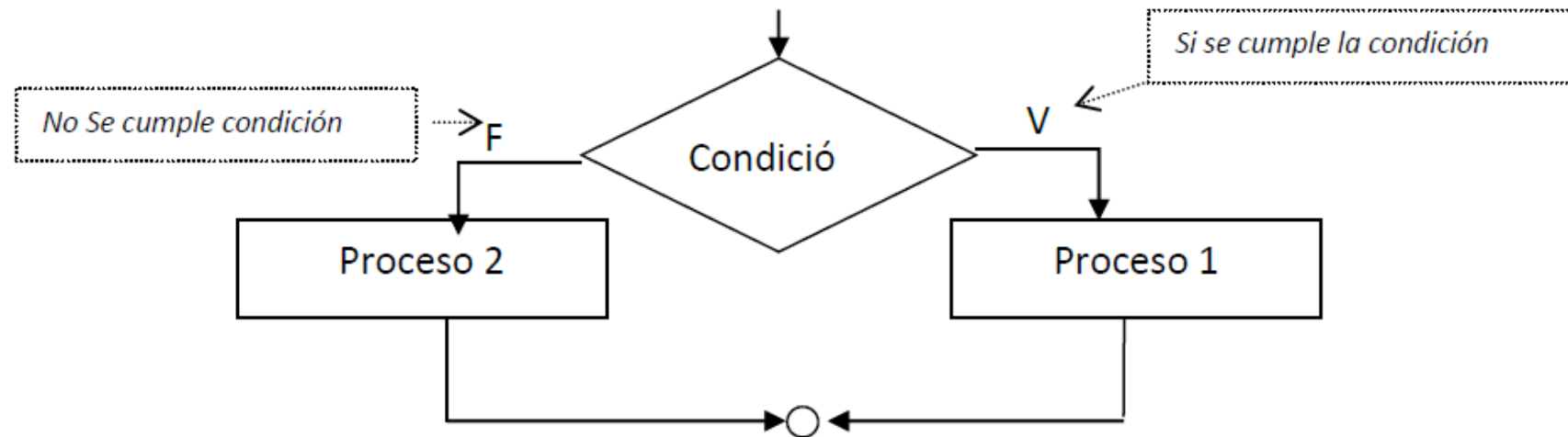
Condicional: `if`

Selección Múltiple: `switch`



Estructuras de Control - Condicional (if)

Estructura lógica en la cual se realiza la división del flujo del proceso de acuerdo a una condición ó pregunta.



Esta estructura se utiliza dentro de un proceso, cuando se presenta una decisión, condición ó pregunta, si se cumple la condición, es decir si es verdadera, se realiza el proceso 1, en caso de no cumplirse la condición, es decir si es falsa se realiza el proceso 2.

Estructuras de Control - Condicional (if)

En pseudocódigo, la estructura del condicional sería:

SI Condición **ENTONCES**

Proceso 1

SINO

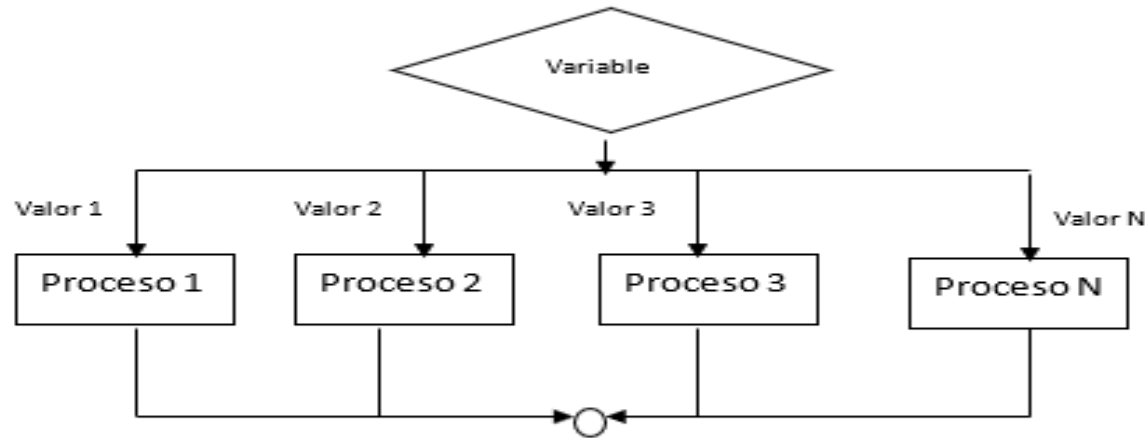
Proceso 2

FIN SI

Estructuras de Control Selección Múltiple (switch)

Selección múltiple (CASE o SWITCH)

Estructura que permite la ejecución de una tarea o proceso dependiendo del valor de una variable, permite varias opciones o ramificaciones para el diagrama de flujo. Gráficamente sería:



Se tiene los siguientes componentes para esta estructura:

- Una variable que tiene valores predefinidos. (Ej: Sexo, Estado civil, estrato, etc.)
- Proceso 1, Proceso 2, Proceso 3 y Proceso N, que son las tareas o actividades que se ejecutan de acuerdo con cada valor válido de la variable.

Estructuras de Control

Selección Múltiple (switch)

En pseudocódigo:

```
SELECCIONE Variable
    EN CASO DE Valor1: Proceso 1
    EN CASO DE Valor2: Proceso 2
    EN CASO DE Valor3: Proceso 3
    .....
    EN CASO DE Valor N: Proceso N
FIN SELECCION
```

Estructuras de control

Estructuras de Control Condicionales Ejercicio

Dada la siguiente información de un usuario del servicio de agua:

- Documento de identidad
- Estado (A=activo, S=Suspendido)
- Estrato(1,2,3,4,5)

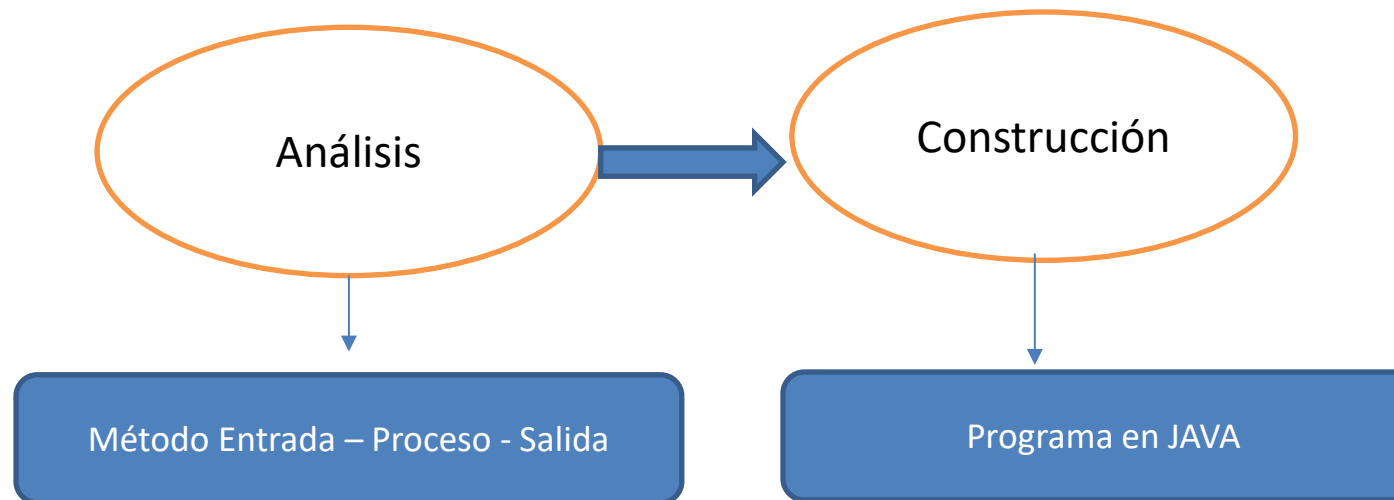
Se pide calcular el valor a pagar por tarifa básica, de acuerdo con las siguientes observaciones:

- Si el usuario está suspendido, el valor de tarifa básica es 0.
- Si el usuario está activo, el valor de la tarifa básica depende del estrato:

Estrato	Tarifa Básica
1	\$10.000
2	\$15.000
3	\$30.000
4	\$50.000
5	\$65.000

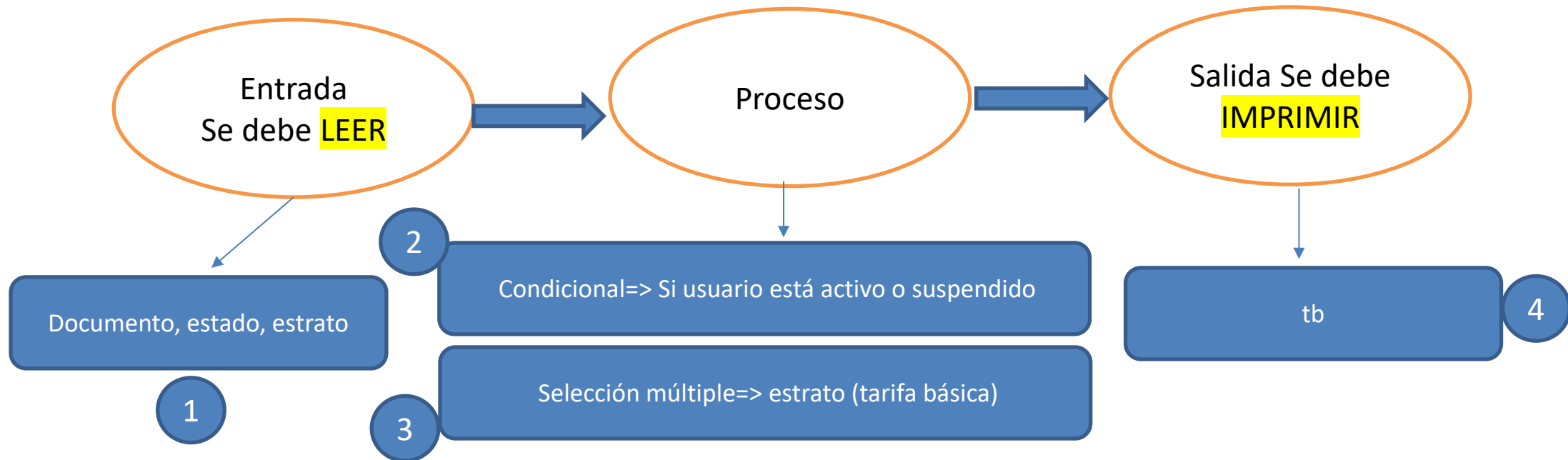
Estructuras de control

Estructuras de Control Condicionales Ejercicio



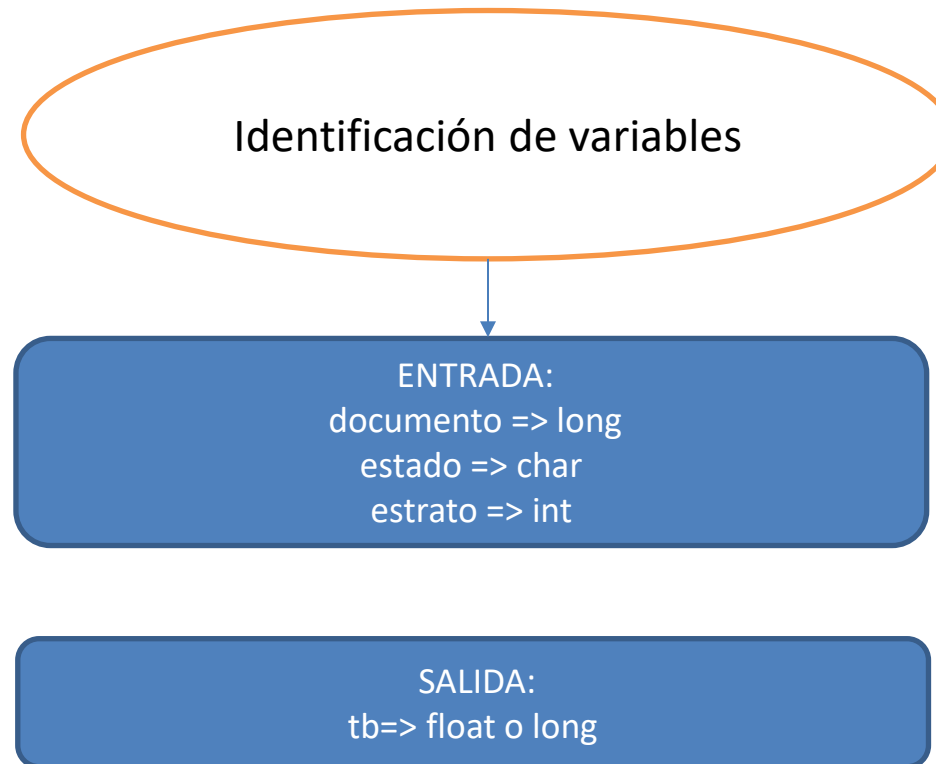
Estructuras de control

Estructuras de Control Condicionales Ejercicio



Estructuras de control

Estructuras de Control Condicionales Ejercicio



Estructuras de control

Calcula_Tarifa_basica - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

<default config>

Projects Files Services

- Calcula_Tarifa_basica
 - Source Packages
 - calcula_tarifa_basica
 - Calcula_Tarifa_basica.java
 - Test Packages
 - Libraries
 - Test Libraries
- JavaApplication1
 - Source Packages
 - Libraries

Navigador

Members

- Calcula_Tarifa_basica
 - main(String[] args)

Start Page JavaApplication1.java Calcula_Tarifa_basica.java

Source History

```
4  * and open the template in the editor.
5  */
6  package calcula_tarifa_basica;
7
8  /**
9   *
10  * @author SERGIO
11  */
12  import java.util.Scanner;
13
14  public class Calcula_Tarifa_basica {
15
16      /**
17       * @param args the command line arguments
18       */
19      public static void main(String[] args) {
20          /* Instancia de Scanner para entrada por consola */
21          Scanner entrada=new Scanner(System.in);
```

calcula_tarifa_basica.Calcula_Tarifa_basica main

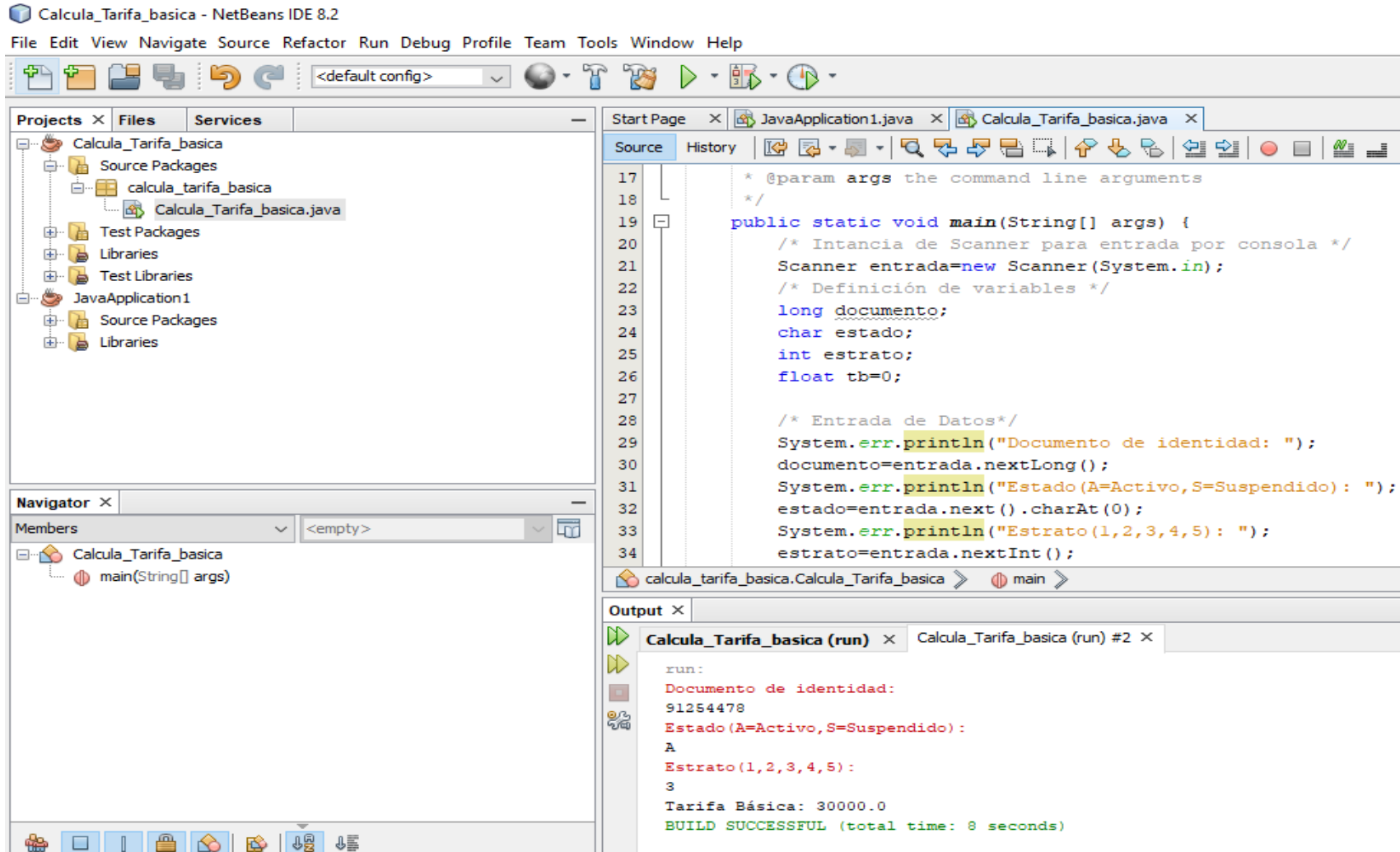
Output

Calcula_Tarifa_basica (run) Calcula_Tarifa_basica (run) #2

```
run:
Documento de identidad:
91254478
Estado (A=Activo,S=Suspendido):
A
Estrato (1,2,3,4,5):
3
Tarifa Básica: 30000.0
BUILD SUCCESSFUL (total time: 8 seconds)
```

Construcción: Programa

Estructuras de control



Construcción: Programa

Estructuras de control

Calcula_Tarifa_basica - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

<default config>

Projects Files Services

- Calcula_Tarifa_basica
 - Source Packages
 - calcula_tarifa_basica
 - Calcula_Tarifa_basica.java
 - Test Packages
 - Libraries
 - Test Libraries
- JavaApplication1
 - Source Packages
 - Libraries

Navigator

Members

- Calcula_Tarifa_basica
 - main(String[] args)

Start Page JavaApplication1.java Calcula_Tarifa_basica.java

Source History

```
35      /*Calcular tarifa básica */
36      if (estado=='S')
37          tb=0;
38      else
39          switch (estrato)
40          {
41              case 1: tb=10000;break;
42              case 2: tb=15000;break;
43              case 3: tb=30000;break;
44              case 4: tb=50000;break;
45              case 5: tb=65000;break;
46          }
47      System.out.println("Tarifa Básica: "+tb);
48
49
50
51
52  }
```

calcula_tarifa_basica.Calcula_Tarifa_basica main

Output

Calcula_Tarifa_basica (run) Calcula_Tarifa_basica (run) #2

```
run:
Documento de identidad:
91254478
Estado (A=Activo, S=Suspendido):
A
Estrato (1,2,3,4,5):
3
Tarifa Básica: 30000.0
BUILD SUCCESSFUL (total time: 8 seconds)
```

Construcción: Programa

Estructuras de Control Iterativas - Ciclos

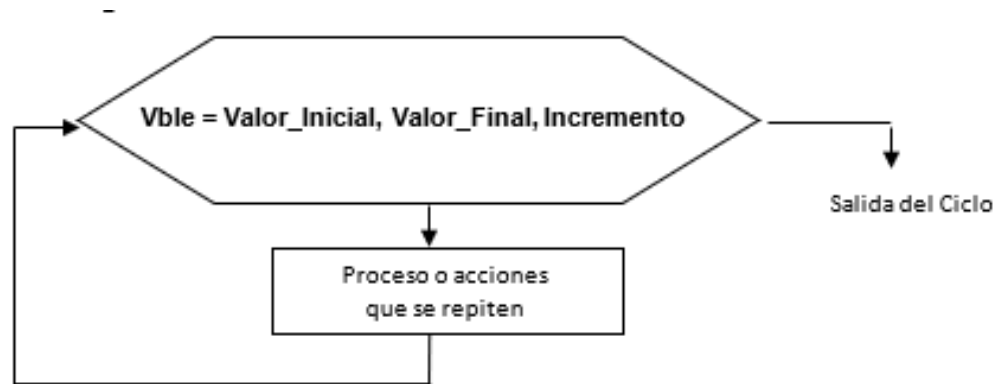
Iterativa controlada por cantidad: **for**

Iterativa controlada por condición: **while**
 do while



Estructuras de control

Iterativa controlada por cantidad FOR



Vble = Encargada de tomar valores de acuerdo con cada ocurrencia de la repetición. Puede darse cualquier nombre a la variable (tener en cuenta las reglas para identificadores), aunque se utiliza con mucha frecuencia, los nombres de I, J, K entre otros. Las iteraciones estarán controladas por esta variable.

Valor_Inicial = Constante o variable con el valor inicial del ciclo, esta asignación solo se realiza una vez, cuando se inicializa el ciclo, es la primera ocurrencia.

Valor_Final = Constante o variable con el valor final del ciclo, esta determina la condición para terminar el ciclo, es la última ocurrencia.

Incremento = Constante o variable que representa el intervalo de aumento de la variable del ciclo, cuando el incremento es 1 se puede omitir, si es un decremento se representa con un valor negativo.

Estructuras de control

Iterativa controlada por cantidad FOR

En pseudocódigo, la estructura sería:

```
PARA Vble= Valor_Inicial HASTA Valor_Final INCREMENTO INCR  
    Proceso que se Repite  
FIN PARA
```

Estructuras de control - Ejercicio

Iterativa controlada por cantidad FOR

Dada la información sobre los **N vendedores**:

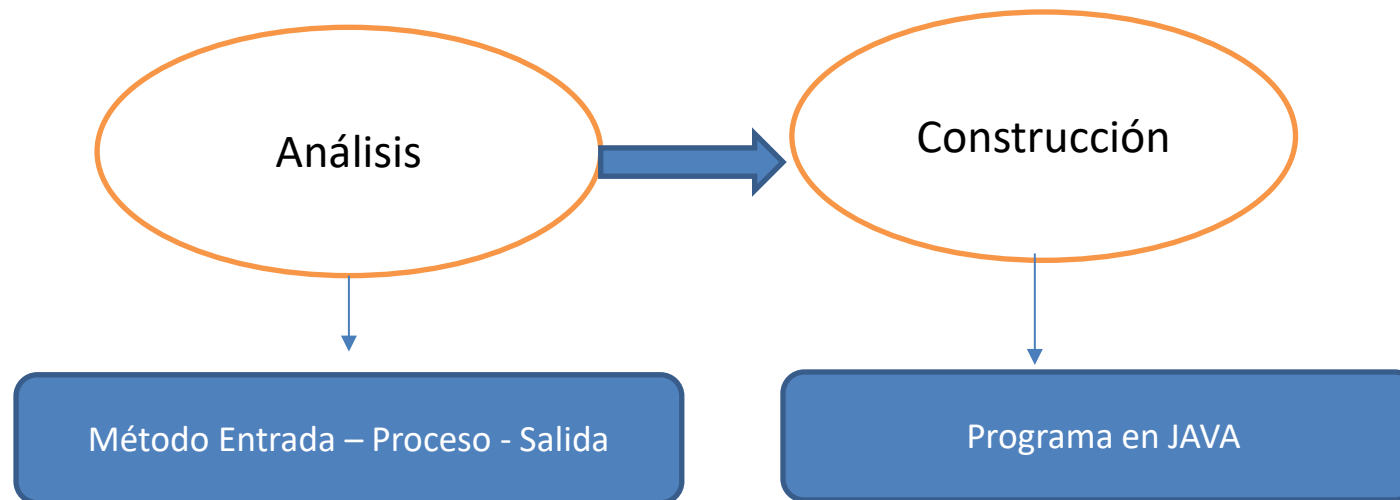
- **Cédula**
- **Tipo Vendedor** (1: Puerta a Puerta 2=Ejecutivo de ventas)
- **Valor ventas del mes**

Se pide calcular el **valor de comisiones** de cada vendedor, de acuerdo con las siguientes observaciones:

Información para liquidar comisiones: Si el vendedor es puerta a puerta (1) gana por comisiones el 20% del valor de sus ventas y si es ejecutivo de ventas (2) gana por comisiones el 30% del valor de sus ventas del mes

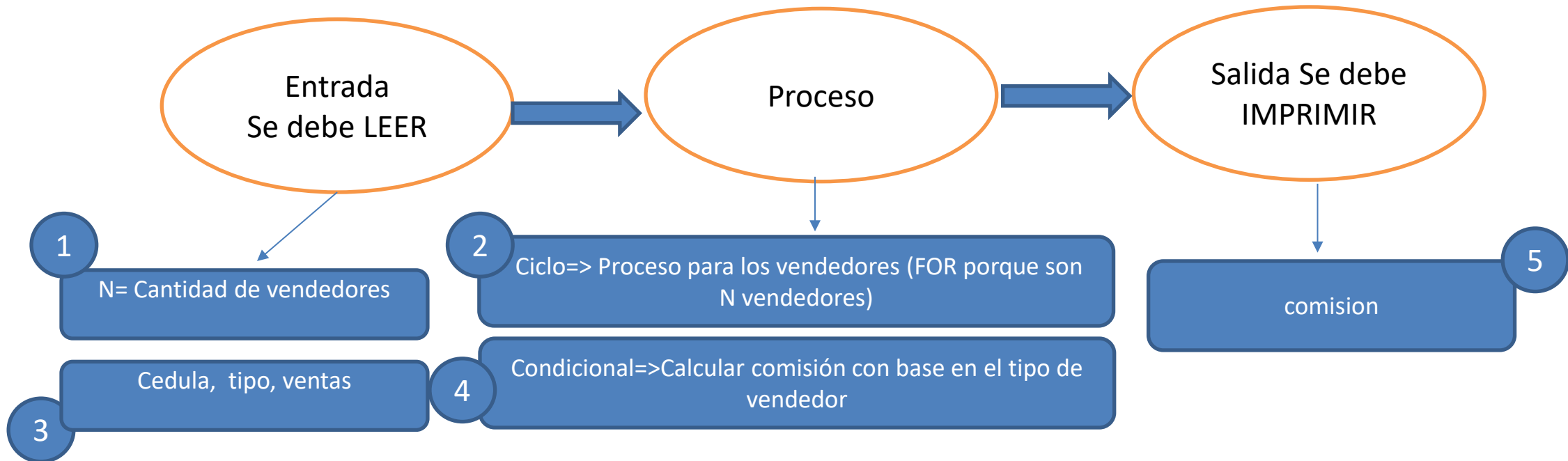
Estructuras de control - Ejercicio

Iterativa controlada por cantidad FOR



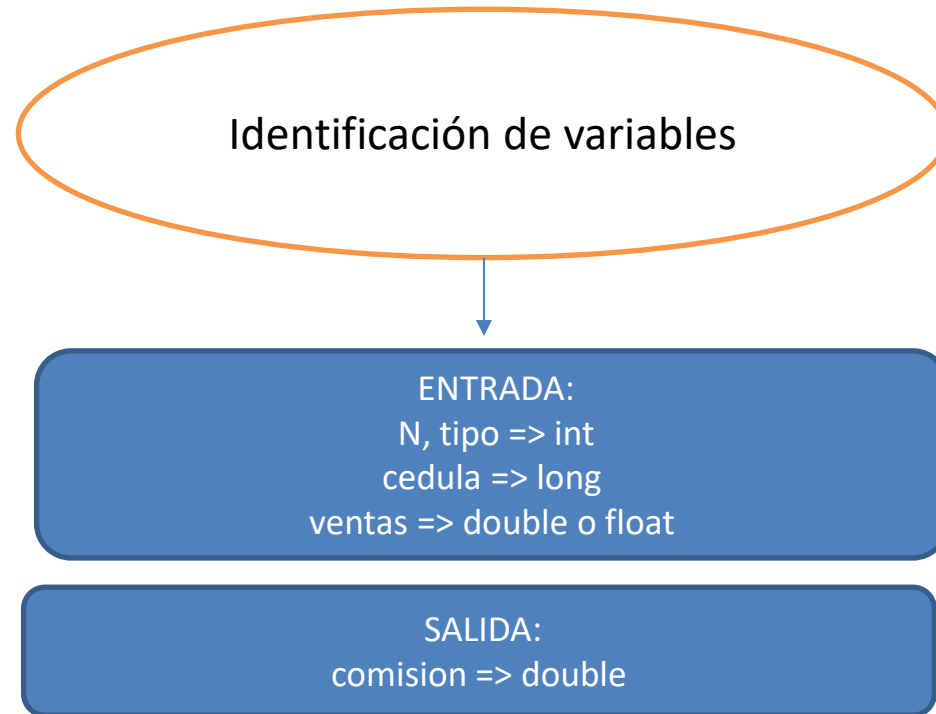
Estructuras de control - Ejercicio

Iterativa controlada por cantidad FOR



Estructuras de control - Ejercicio

Iterativa controlada por cantidad FOR



Estructuras de control - Ejercicio

calculo_comisiones - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

<default config>

Projects Files Services

- calculo_comisiones
 - Source Packages
 - calculo_comisiones
 - Calculo_comisiones.java
 - Test Packages
 - Libraries
 - Test Libraries
- JavaApplication 1
 - Source Packages
 - javaapplication1
 - JavaApplication1.java
 - Libraries

main - Navigator

Members

- Calculo_comisiones
 - main(String[] args)

Start Page JavaApplication1.java Calculo_comisiones.java

Source History

```
5  /**
6   package calculo_comisiones;
7
8   /**
9    *
10   * @author SERGIO
11   */
12  import java.util.Scanner;
13
14  public class Calculo_comisiones {
15
16      /**
17       * @param args the command line arguments
18       */
19      public static void main(String[] args) {
20          /* Instancia Scanner para lectura por consola */
21          Scanner entrada=new Scanner(System.in);
22          /*Definición de variables */
23          long cedula;
```

calculo_comisiones.Calculo_comisiones main

Output

Calcula_Tarifa_basica (run) calculo_comisiones (run)

```
Comisión: 200000.0
Cédula:
2
Tipo(1=Puerta a Puerta, 2=Ejecuyivo):
2
Valor ventas:
3000000
Comisión: 900000.0
BUILD SUCCESSFUL (total time: 19 seconds)
```

Estructuras de control - Ejercicio

calculo_comisiones - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Projects Files Services

- calculo_comisiones
 - Source Packages
 - calculo_comisiones
 - Calculo_comisiones.java
 - Test Packages
 - Libraries
 - Test Libraries
- JavaApplication1
 - Source Packages
 - javaapplication1
 - JavaApplication1.java
 - Libraries

main - Navigator

Members

- Calculo_comisiones
 - main(String[] args)

Source History

```
20      /* Instancia Scanner para lectura por consola */
21      Scanner entrada=new Scanner(System.in);
22      /*Definición de variables */
23      long cedula;
24      int N,tipo;
25      double comision,ventas;
26      /* Entrada de datos */
27      System.err.println("Cantidad de vebdedores: ");
28      N=entrada.nextInt();
29      /* Ciclo para calacular comisiones */
30      for (int i=1;i<=N;i++)
31      {
32          System.err.println("Cédula: ");
33          cedula=entrada.nextLong();
34          System.err.println("Tipo(1=Puerta a Puerta, 2=Ejecuyivo): ");
35          tipo=entrada.nextInt();
36          System.err.println("Valor ventas: ");
37          ventas=entrada.nextDouble();
```

calculo_comisiones.Calculo_comisiones main

Output

Calcula_Tarifa_basica (run) calculo_comisiones (run)

```
Comisión: 200000.0
Cédula:
2
Tipo(1=Puerta a Puerta, 2=Ejecuyivo):
2
Valor ventas:
3000000
Comisión: 900000.0
BUILD SUCCESSFUL (total time: 19 seconds)
```

Estructuras de control - Ejercicio

calculo_comisiones - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Projects Files Services

- calculo_comisiones
 - Source Packages
 - calculo_comisiones
 - Calculo_comisiones.java
 - Test Packages
 - Libraries
 - Test Libraries
- JavaApplication1
 - Source Packages
 - javaapplication1
 - JavaApplication1.java
 - Libraries

main - Navigator

Members

- Calculo_comisiones
 - main(String[] args)

Source History

```
30 for (int i=1;i<=N;i++)
31 {
32     System.err.println("Cédula: ");
33     cedula=entrada.nextLong();
34     System.err.println("Tipo(1=Puerta a Puerta, 2=Ejecutivo): ");
35     tipo=entrada.nextInt();
36     System.err.println("Valor ventas: ");
37     ventas=entrada.nextDouble();
38     if (tipo==1)
39         comision=ventas*0.20;
40     else
41         comision=ventas*0.30;
42     System.out.println("Comisión: "+comision);
43 }
44 }
45 }
46 }
47 }
```

calculo_comisiones.Calculo_comisiones > main >

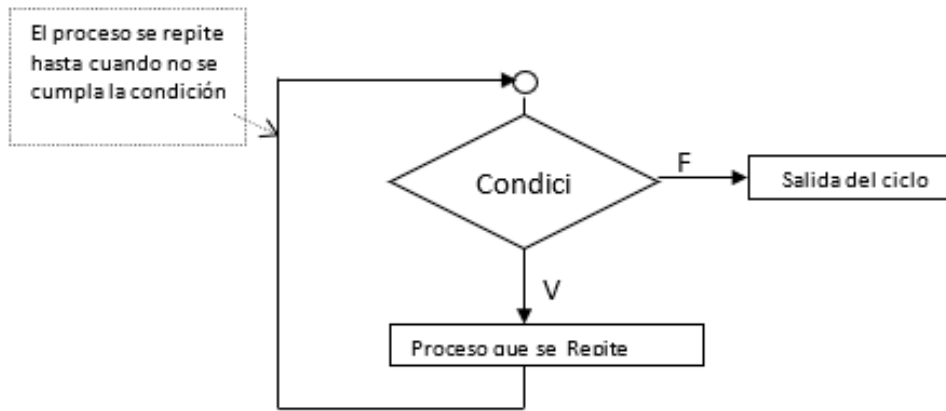
Output

Calcula_Tarifa_basica (run) x calculo_comisiones (run) x

```
Comisión: 200000.0
Cédula:
2
Tipo(1=Puerta a Puerta, 2=Ejecutivo):
2
Valor ventas:
3000000
Comisión: 900000.0
BUILD SUCCESSFUL (total time: 19 seconds)
```

Estructuras de control

Iterativa controlada por condición WHILE



Una característica importante de esta estructura es que está conformada de dos elementos fundamentales:

- Una Condición
- Un Proceso que se repite

Y además se tiene como norma que PRIMERO se realiza la condición y SI SE CUMPLE entonces se realiza como SEGUNDA medida el proceso. Por ello esta estructura se debe utilizar cuando se tengan las siguientes características:

1. Debe existir un proceso que se repita (CICLO).
2. La repetición del proceso debe depender de una condición.

Estructuras de control

Iterativa controlada por condición **WHILE**

En pseudocódigo:

```
MIENTRAS Condición HAGA  
    Proceso que se repite  
FIN MIENTRAS
```

Estructuras de control

Iterativa controlada por condición WHILE

GUIA del WHILE (Iteración manejada con condición)

En la estructura de control WHILE se presenta una DOBLE lectura de información, por las siguientes razones:

- La primera lectura se encuentra ANTES del ciclo, debido a que, al ingreso al ciclo WHILE, se encuentra una condición que necesita tener un valor en la variable de control bandera.
- La segunda lectura se encuentra DENTRO del ciclo, al final del proceso que se repite, debido a que es necesario, una vez procesada una ocurrencia de la repetición, pasar a la siguiente información almacenada en la variable bandera, es decir, pasar a leer el siguiente elemento. Si no se realiza esta lectura del siguiente elemento se podría incurrir en un ciclo SIN FIN, por no tener variación en la variable de control necesaria para el ingreso o salida del ciclo

Estructuras de control

Iterativa controlada por condición WHILE

Dada la información sobre una lista de vendedores:

- Cédula
- Tipo Vendedor (1: Puerta a Puerta 2=Ejecutivo de ventas)
- Valor ventas del mes

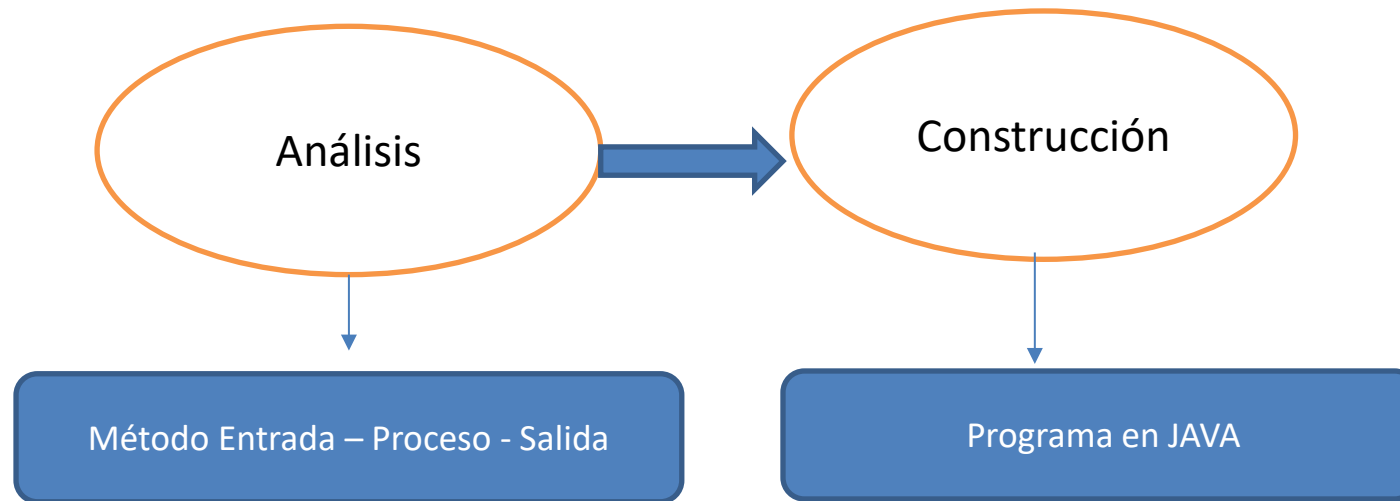
La lista termina cuando la cédula ingresada sea 999

Se pide calcular el valor de comisiones de cada vendedor, de acuerdo con las siguientes observaciones:

Información para liquidar comisiones: Si el vendedor es puerta a puerta (1) gana por comisiones el 20% del valor de sus ventas y si es ejecutivo de ventas (2) gana por comisiones el 30% del valor de sus ventas del mes

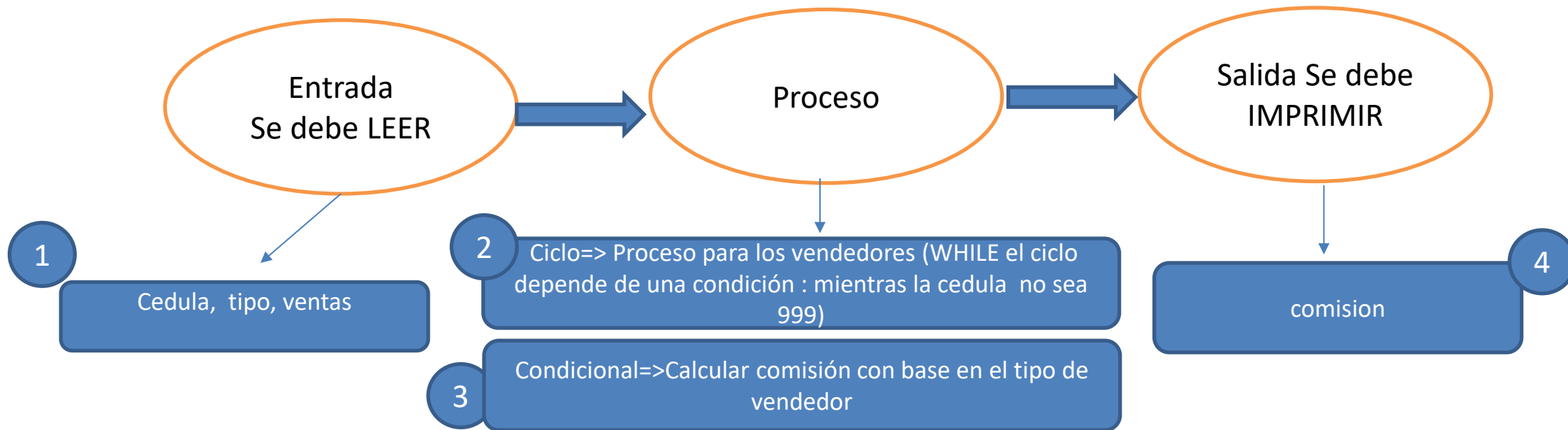
Estructuras de control - Ejercicio

Iterativa controlada por condición
WHILE



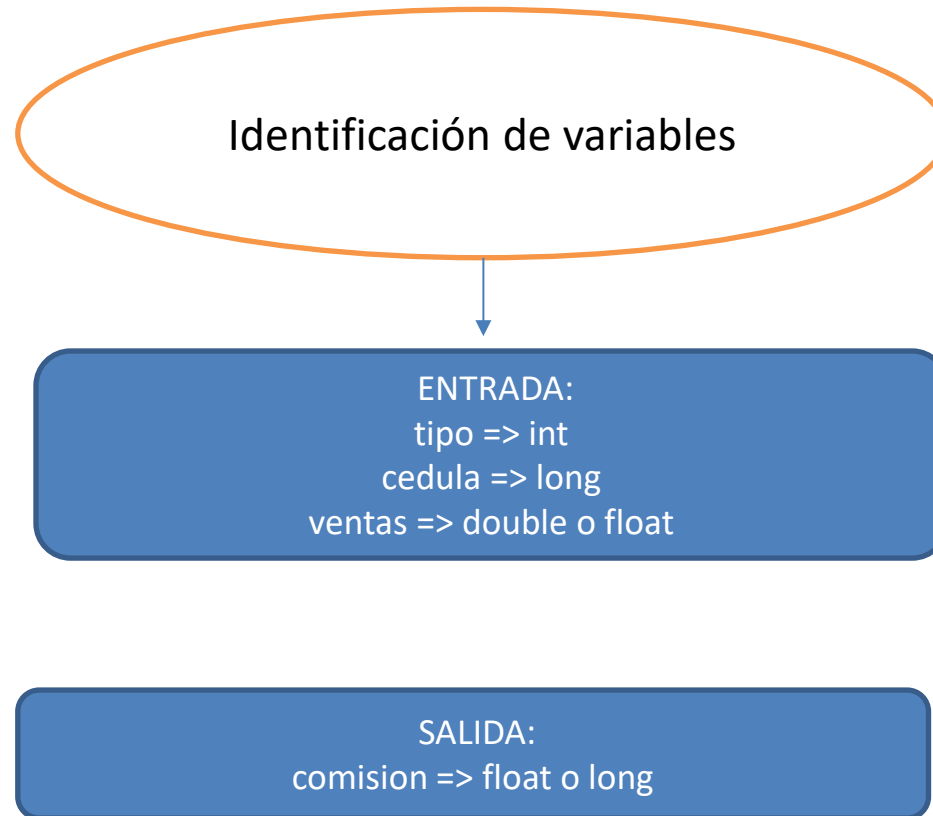
Estructuras de control

Iterativa controlada por condición WHILE

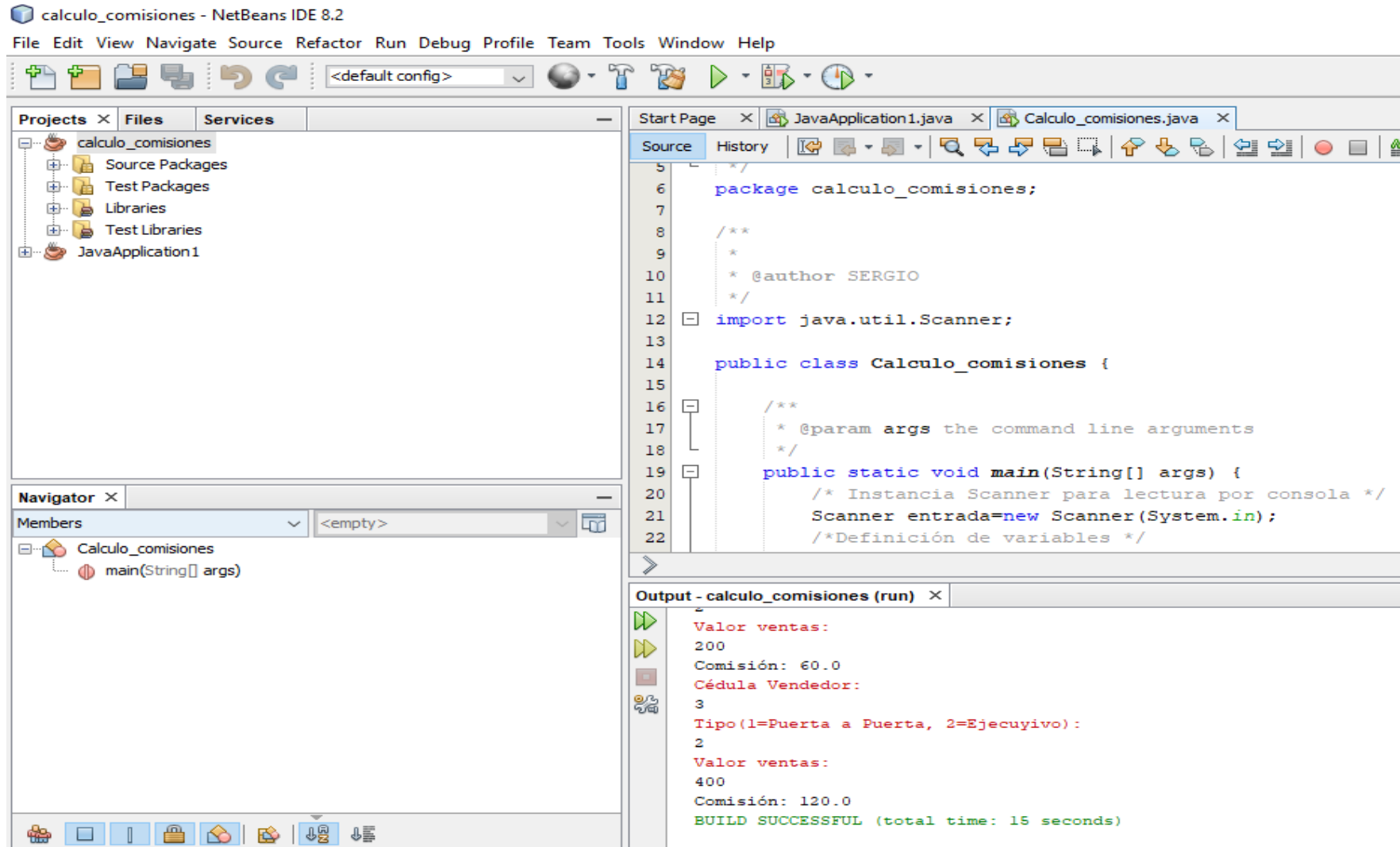


Estructuras de control

Iterativa controlada por condición WHILE



Estructuras de control



Estructuras de control

calculo_comisiones - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Projects Files Services

- calculo_comisiones
 - Source Packages
 - Test Packages
 - Libraries
 - Test Libraries
 - JavaApplication1

Start Page JavaApplication1.java Calculo_comisiones.java

Source History

```
22  /* Definición de variables */
23  long cedula;
24  int tipo;
25  double comision, ventas;
26  /* Leer primer vendedor para entrar al while */
27  System.err.println("Cédula Vendedor: ");
28  cedula=entrada.nextInt();
29  /* Ciclo para calcular comisiones */
30  while (cedula!=999)
31  {
32      System.err.println("Tipo (1=Puerta a Puerta, 2=Ejecutivo): ");
33      tipo=entrada.nextInt();
34      System.err.println("Valor ventas: ");
35      ventas=entrada.nextDouble();
36      if (tipo==1)
37          comision=ventas*0.20;
38      else
39          comision=ventas*0.30;
```

Navigador

Members

- Calculo_comisiones
 - main(String[] args)

Output - calculo_comisiones (run)

```
Valor ventas:
200
Comisión: 60.0
Cédula Vendedor:
3
Tipo (1=Puerta a Puerta, 2=Ejecutivo):
2
Valor ventas:
400
Comisión: 120.0
BUILD SUCCESSFUL (total time: 15 seconds)
```

Estructuras de control

calculo_comisiones - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Projects Files Services

- calculo_comisiones
 - Source Packages
 - Test Packages
 - Libraries
 - Test Libraries
 - JavaApplication1

Start Page JavaApplication1.java Calculo_comisiones.java

Source History

```
30 while (cedula!=999)
31 {
32     System.err.println("Tipo(1=Puerta a Puerta, 2=Ejecuyivo): ");
33     tipo=entrada.nextInt();
34     System.err.println("Valor ventas: ");
35     ventas=entrada.nextDouble();
36     if (tipo==1)
37         comision=ventas*0.20;
38     else
39         comision=ventas*0.30;
40     System.out.println("Comisión: "+comision);
41     /*Leer siguiente */
42     System.err.println("Cédula: ");
43     cedula=entrada.nextLong();
44 }
45 }
46
47
```

Navigator

Members

- Calculo_comisiones
 - main(String[] args)

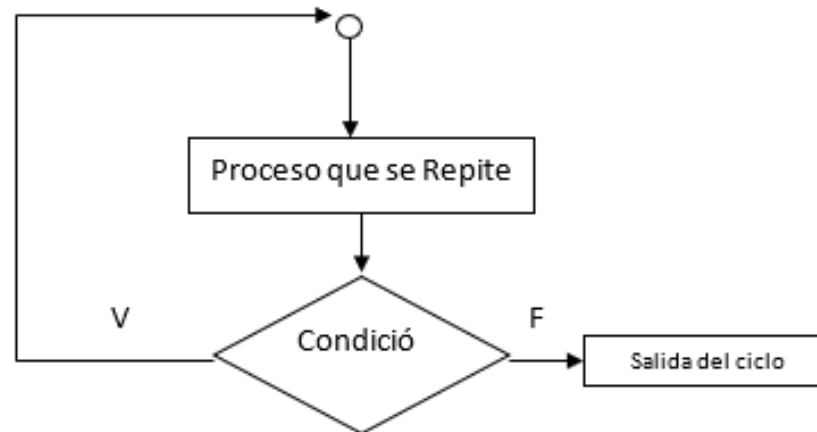
Output - calculo_comisiones (run)

```
Valor ventas:
200
Comisión: 60.0
Cédula Vendedor:
3
Tipo(1=Puerta a Puerta, 2=Ejecuyivo):
2
Valor ventas:
400
Comisión: 120.0
BUILD SUCCESSFUL (total time: 15 seconds)
```

Estructuras de control

☑ Repetición Condicional - HACER MIENTRAS (DO - WHILE)

Estructura en donde aparece nuevamente un proceso que se repite, CICLO, pero esta vez su repetición depende del cumplimiento de una condición, es similar a la estructura MIENTRAS se diferencia en que la condición va después del proceso, es decir este bucle siempre realiza por lo menos una iteración antes de evaluar la condición. Gráficamente sería:



En esta estructura el proceso se repite mientras la condición sea verdadera.

Estructuras de control

Iterativa controlada por condición DO WHILE

En pseudocódigo:

HACER

Proceso que se repite

MIENTRAS Condición

Estructuras de control

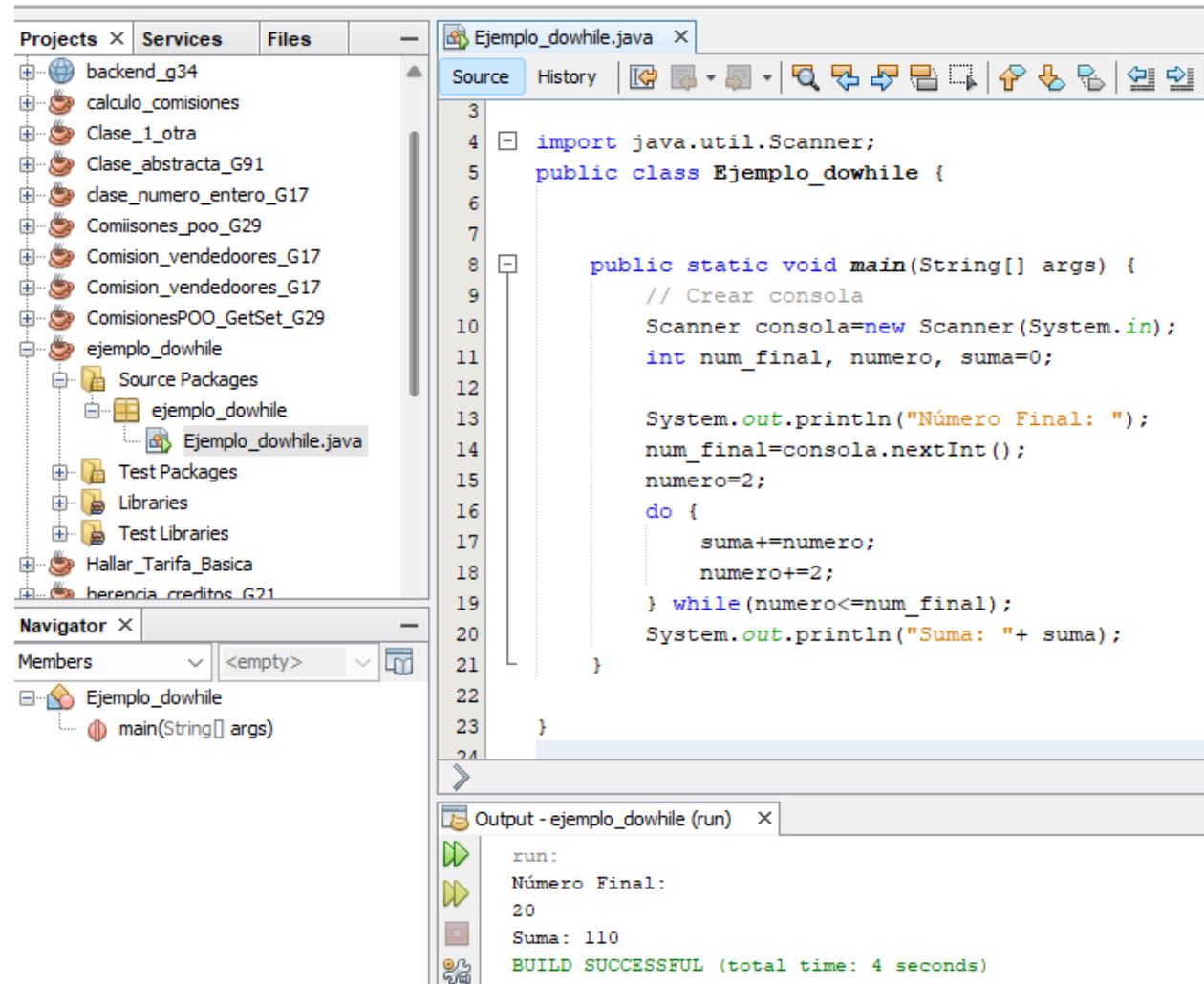
Iterativa controlada por condición DO WHILE

Se pide calcular e imprimir la suma de los números pares desde 2 hasta un número final que se ingresa por consola

Estructuras de control

Iterativa controlada por condición

DO WHILE



The screenshot shows an IDE with a project named 'ejemplo_dowhile'. The 'Source' tab is active, displaying the following Java code:

```
3
4 import java.util.Scanner;
5 public class Ejemplo_dowhile {
6
7
8     public static void main(String[] args) {
9         // Crear consola
10        Scanner consola=new Scanner(System.in);
11        int num_final, numero, suma=0;
12
13        System.out.println("Número Final: ");
14        num_final=consola.nextInt();
15        numero=2;
16        do {
17            suma+=numero;
18            numero+=2;
19        } while(numero<=num_final);
20        System.out.println("Suma: "+ suma);
21    }
22
23 }
```

The 'Navigator' panel on the left shows the project structure, including 'Source Packages', 'Test Packages', 'Libraries', and 'Test Libraries'. The 'Members' panel shows the 'main(String[] args)' method.

The 'Output - ejemplo_dowhile (run)' panel at the bottom shows the execution results:

```
run:
Número Final:
20
Suma: 110
BUILD SUCCESSFUL (total time: 4 seconds)
```

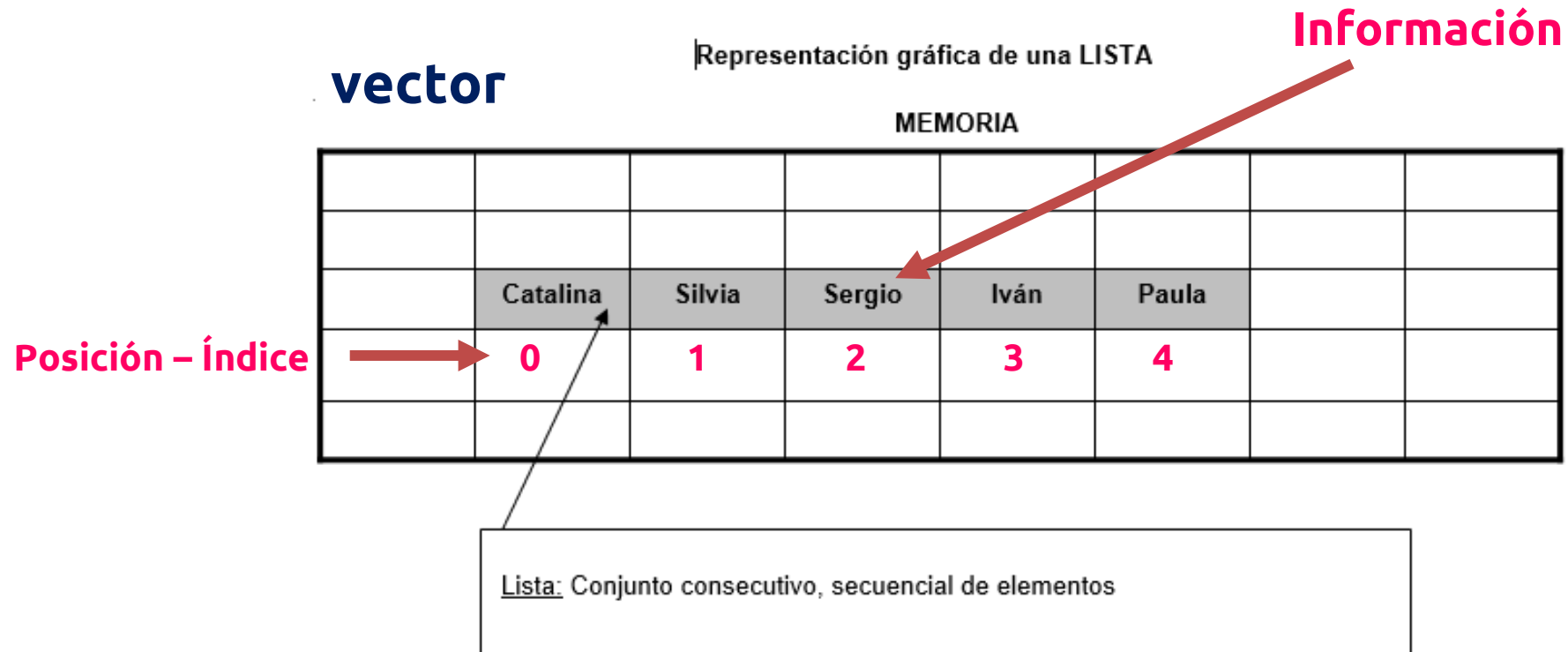
Listas – Arreglos

Vectores



Estructuras de Datos

Arreglos -Vector



Estructuras de Datos

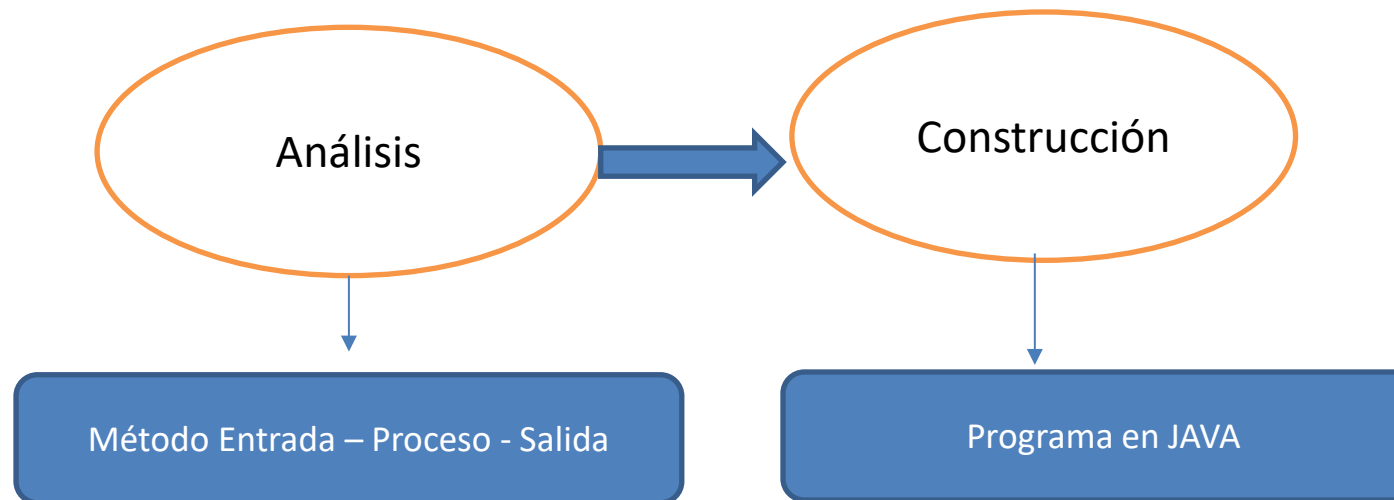
Arreglos -Vector

Dados 6 números enteros que se almacenan en un **vector**, calcular:

- **Cuáles y cuantos son pares**
- **Cuáles y cuantos son impares**
- **La suma de los números pares**
- **La suma de los números impares**

Estructuras de Datos

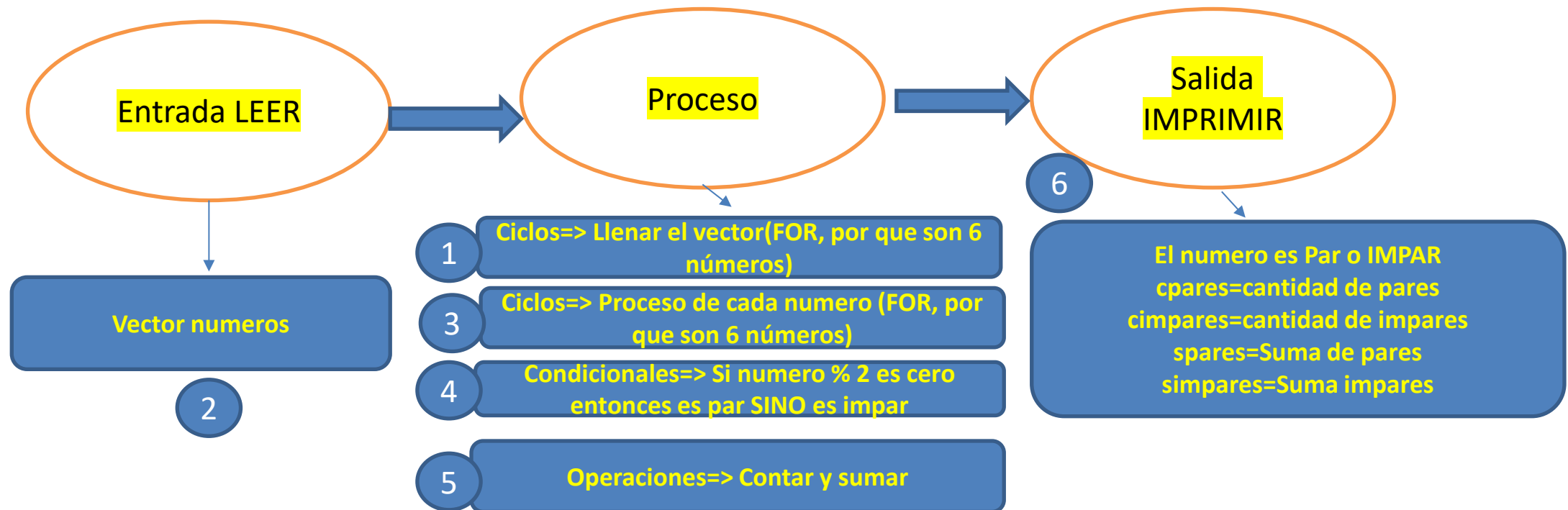
Arreglos -Vector



Estructuras de Datos

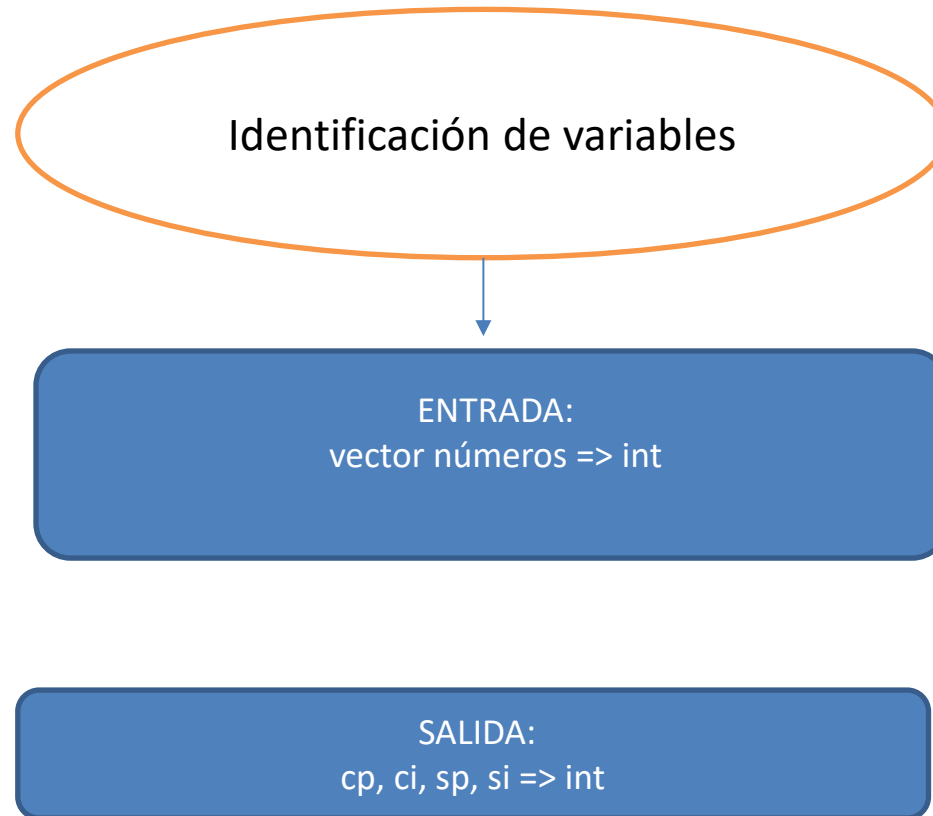
Arreglos -Vector

Análisis → Ejercicio pares e impares



Estructuras de Datos

Arreglos -Vector



Arreglos -Vector

The screenshot displays the NetBeans IDE 8.2 environment. The title bar indicates the active project is 'vectores_java'. The menu bar includes File, Edit, View, Navigate, Source, Refactor, Run, Debug, Profile, Team, Tools, Window, and Help. The toolbar contains various icons for file operations and development actions.

The left sidebar shows the 'Projects' view with a tree structure: JavaApplication1 (Source Packages, javaapplication1, JavaApplication1.java), Libraries, vectores_java (Source Packages, vectores_java, Vectores_java.java), Test Packages, Libraries, and Test Libraries. The 'main - Navigator' view shows the 'Members' of the 'Vectores_java' package, listing the 'main(String[] args)' method.

The central editor window displays the source code of 'Vectores_java.java'. The code includes package declarations, imports, and a public class 'Vectores_java' with a 'main' method. The 'main' method uses a 'Scanner' to read input and calculates the sum of even and odd numbers. The code is as follows:

```
5  /*
6  package vectores_java;
7
8  /**
9   *
10  * @author SERGIO
11  */
12  import java.util.Scanner;
13
14  public class Vectores_java {
15
16      /**
17       * @param args the command line arguments
18       */
19      public static void main(String[] args) {
20          /* Instancia Scanner para entrada de datos por consola */
21          Scanner entrada=new Scanner(System.in);
22          /* Definición de variables */
```

The bottom 'Output' window shows the execution results of the 'Calcula_Tarifa_basica (run)' and 'vectores_java (run)' tasks. The output displays the results of the program's execution, including the number of even and odd numbers and their respective sums.

```
Calcula_Tarifa_basica (run) x vectores_java (run) x
El número es IMPAR : 3
El número es PAR : 4
El número es IMPAR : 5
El número es PAR : 6
Cantidad de pares : 3
Cantidad de impares : 3
Suma de pares : 12
Suma de pares : 9
BUILD SUCCESSFUL (total time: 8 seconds)
```

Estructuras de Datos

Arreglos -Vector

vector.es_java - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Projects Files Services

- JavaApplication1
 - Source Packages
 - javaapplication1
 - JavaApplication1.java
 - Libraries
- vector.es_java
 - Source Packages
 - vector.es_java
 - Vector.es_java.java
 - Test Packages
 - Libraries
 - Test Libraries

main - Navigator

Members

- Vector.es_java
 - main(String[] args)

Source History

```
22  /* Definición de variables */
23  long cp,ci,sp,si;
24  long numeros[];
25  numeros=new long[10];
26  for (int i=1;i<=6;i++)
27  {
28      System.out.println("Ingreso número: ");
29      numeros[i]=entrada.nextLong();
30
31  }
32  cp=0;ci=0;sp=0;si=0;
33  for (int i=1;i<=6;i++)
34  {
35      if (numeros[i] % 2==0)
36      {
37          System.out.println("El número es PAR : "+numeros[i]);
38          cp=cp+1;
39          sp=sp+numeros[i];
```

vector.es_java.Vector.es_java > main > for (inti = 1; i <= 6; i++) >

Output

Calcula_Tarifa_basica (run) x vector.es_java (run) x

```
El número es IMPAR : 3
El número es PAR : 4
El número es IMPAR : 5
El número es PAR : 6
Cantidad de pares : 3
Cantidad de impares : 3
Suma de pares : 12
Suma de impares : 9
BUILD SUCCESSFUL (total time: 8 seconds)
```

Estructuras de Datos

Arreglos -Vector

vectores_java - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Projects Files Services

- JavaApplication1
 - Source Packages
 - javaapplication1
 - JavaApplication1.java
 - Libraries
- vectores_java
 - Source Packages
 - vectores_java
 - Vectores_java.java
 - Test Packages
 - Libraries
 - Test Libraries

main - Navigator

Members

- Vectores_java
 - main(String[] args)

Source

```
35         if (numeros[i] % 2==0)
36         {
37             System.out.println("El número es PAR : "+numeros[i]);
38             cp=cp+1;
39             sp=sp+numeros[i];
40         }
41         else
42         {
43             System.out.println("El número es IMPAR : "+numeros[i]);
44             ci=ci+1;
45             si=si+numeros[i];
46         }
47
48
49     }
50     System.out.println("Cantidad de pares : "+cp);
51     System.out.println("Cantidad de impares : "+ci);
52     System.out.println("Suna de pares : "+sp);
53     System.out.println("Suma de impares : "+si);
54 }
55
56 for (int i = 1; i <= 6; i++)
```

Output

Calcula_Tarifa_basica (run) x vectores_java (run) x

```
El número es IMPAR : 3
El número es PAR : 4
El número es IMPAR : 5
El número es PAR : 6
Cantidad de pares : 3
Cantidad de impares : 3
Suna de pares : 12
Suma de pares : 9
BUILD SUCCESSFUL (total time: 8 seconds)
```

Funciones

Modularidad

Acoplamiento y Cohesión de módulos

Funciones en JAVA



Funciones

Modularidad

Uno de los aspectos fundamentales de la programación moderna, base de los nuevos paradigmas, es sin duda alguna la **modularidad**, entendida como la generación de módulos o segmentos funcionales e independientes que permitan una mejor organización y compresión de un programa. Este aspecto se basa en la aplicación de dos técnicas propias de la ingeniería del software, denominadas **Acoplamiento de módulos** y **Cohesión de módulos** que definen unas guías en la definición de un módulo.

Funciones

Cohesión de módulos

La técnica de la ingeniería del software, denominada Cohesión de Módulos busca medir el grado de **relación** ò **dependencia** que existe entra las **actividades propias** de un **proceso o módulo**.

Acoplamiento de módulos

La técnica La técnica del Acoplamiento de Módulos que se aplica después de la cohesión, tiene como objetivo la generación de **módulos independientes** dentro de un proceso, en los cuales, **cada uno de ellos** **defina sus propias variables** y **la comunicación entre ellos se haga a través de parámetros**, o sea, variables que un módulo envía y el otro recibe en variables propias

Funciones

Ejercicio

La empresa de teléfonos de la ciudad necesita realizar su proceso de facturación en forma automática, contando con los **N abonados**, de los cuales conoce el **código**, **estrato**, que puede ser (1, 2, 3, 4, 5), cantidad de **impulsos del mes**. Además la empresa nos informa que para la liquidación de la factura se debe tener en cuenta el valor de la tarifa básica, de acuerdo al estrato, que depende de la siguiente tabla:

Estrato	tarifa Básica
1	\$10.000
2	\$15.000
3	\$20.000
4	\$25.000
5	\$30.000

Además se debe calcular el valor de los impulsos, con base en la cantidad de impulsos del mes, conociendo que cada impulso tiene un valor de \$100. Con esta información, se desea:

- ☒ Valor a pagar de cada abonado.
- ☒ Valor total a pagar(Todos los abonados)

Funciones

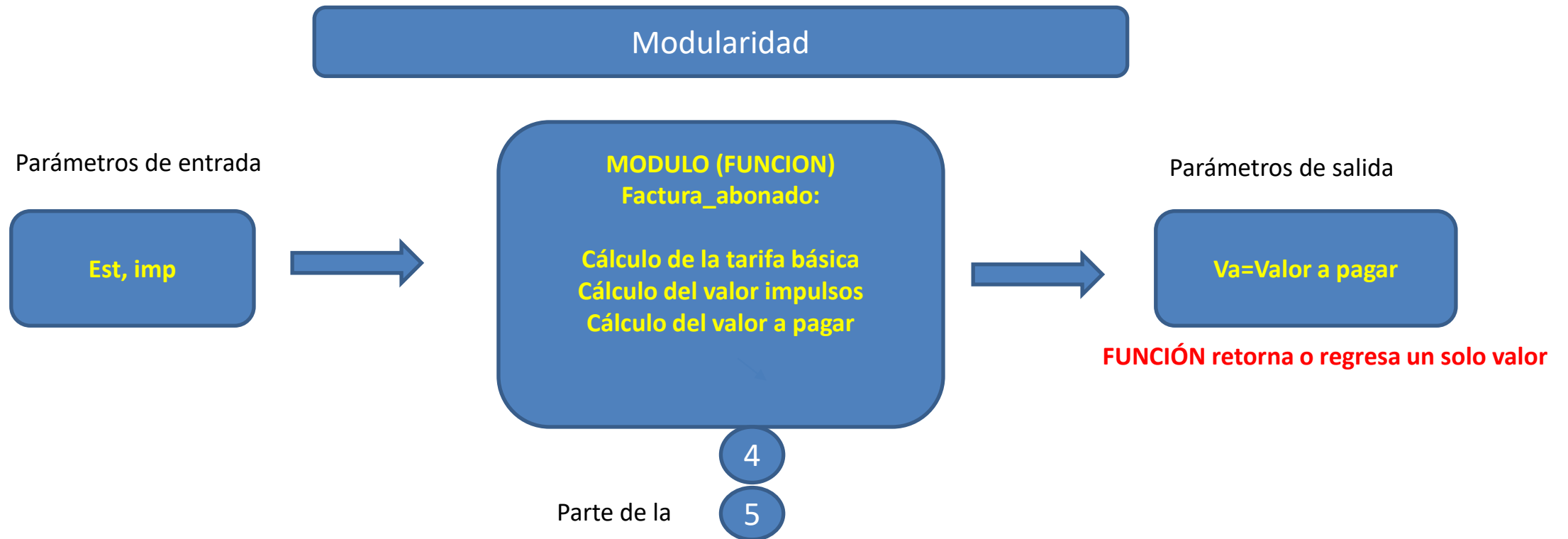
Ejercicio

Análisis → Ejercicio funciones



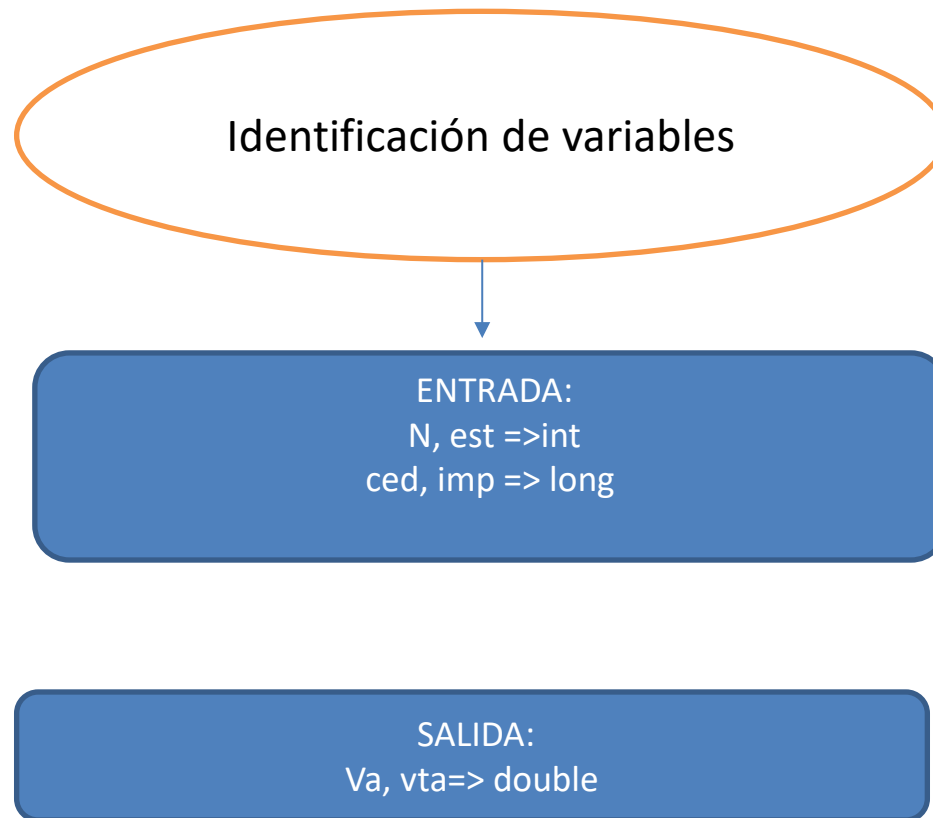
Funciones

Ejercicio



Funciones

Ejercicio



Funciones

facturacion_telefono_funciones - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Projects Files Services

- facturacion_telefono_funciones
 - Source Packages
 - facturacion_telefono_funciones
 - Facturacion_telefono_funciones.java
 - Test Packages
 - Libraries
 - Test Libraries
- JavaApplication1

main - Navigator

Members

- Facturacion_telefono_funciones
 - factura_abonado(int estrato, long impulsos) : double
 - main(String[] args)

Start Page JavaApplication1.java Facturacion_telefono_funciones.java

Source History

```
6 package facturacion_telefono_funciones;
7
8 /**
9  *
10  * @author SERGIO
11  */
12 import java.util.Scanner;
13
14 public class Facturacion_telefono_funciones {
15
16     /**
17      * @param args the command line arguments
18      */
19     static double factura_abonado(int estrato, long impulsos)
20     {
21         double tb=0, va;
22         switch (estrato)
23         {
```

facturacion_telefono_funciones.Facturacion_telefono_funciones main for (inti = 1; i <= N; i++)

Output - facturacion_telefono_funciones (run)

```
Impulsos mes:
100
Valor factura: 20000.0
Cedula:
2
Estrato(1,2,3,4,5):
4
Impulsos mes:
200
Valor factura: 45000.0
Valor total facturación: 65000.0
BUILD SUCCESSFUL (total time: 15 seconds)
```

Funciones

facturacion_telefono_funciones - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

<default config>

Projects Files Services

- facturacion_telefono_funciones
 - Source Packages
 - facturacion_telefono_funciones
 - Facturacion_telefono_funciones.java
 - Test Packages
 - Libraries
 - Test Libraries
 - JavaApplication 1

main - Navigator

Members

- Facturacion_telefono_funciones
 - factura_abonado(int estrato, long impulsos) : double
 - main(String[] args)

Source History

```
16  /**
17   * @param args the command line arguments
18   */
19   static double factura_abonado(int estrato, long impulsos)
20   {
21       double tb=0, va;
22       switch (estrato)
23       {
24           case 1:tb=10000;break;
25           case 2:tb=15000;break;
26           case 3:tb=20000;break;
27           case 4:tb=25000;break;
28           case 5:tb=30000;break;
29       }
30       va=tb+impulsos*100;
31       return va;
32   }
33 }
```

facturacion_telefono_funciones.Facturacion_telefono_funciones main for (inti = 1; i <= N; i++)

Output - facturacion_telefono_funciones (run)

```
Impulsos mes:
100
Valor factura: 20000.0
Cedula:
2
Estrato(1,2,3,4,5):
4
Impulsos mes:
200
Valor factura: 45000.0
Valor total facturación: 65000.0
BUILD SUCCESSFUL (total time: 15 seconds)
```

Funciones

facturacion_telefono_funciones - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Projects Files Services

- facturacion_telefono_funciones
 - Source Packages
 - facturacion_telefono_funciones
 - Facturacion_telefono_funciones.java
 - Test Packages
 - Libraries
 - Test Libraries
 - JavaApplication1

main - Navigator

Members

- Facturacion_telefono_funciones
 - factura_abonado(int estrato, long impulsos) : double
 - main(String[] args)

Source History

```
42  entrada.nextLine();
43      for (int i=1;i<=N;i++)
44      {
45          System.out.println("Cedula: ");
46          ced=entrada.nextLong();
47          System.out.println("Estrato(1,2,3,4,5): ");
48          est=entrada.nextInt();
49          System.out.println("Impulsos mes: ");
50          imp=entrada.nextLong();
51          // Llamada a la función
52          va=factura_abonado(est,imp);
53          System.out.println("Valor factura: "+va);
54          vta+=va;
55      }
56      System.out.println("Valor total facturación: "+vta);
57  }
58
59  }
```

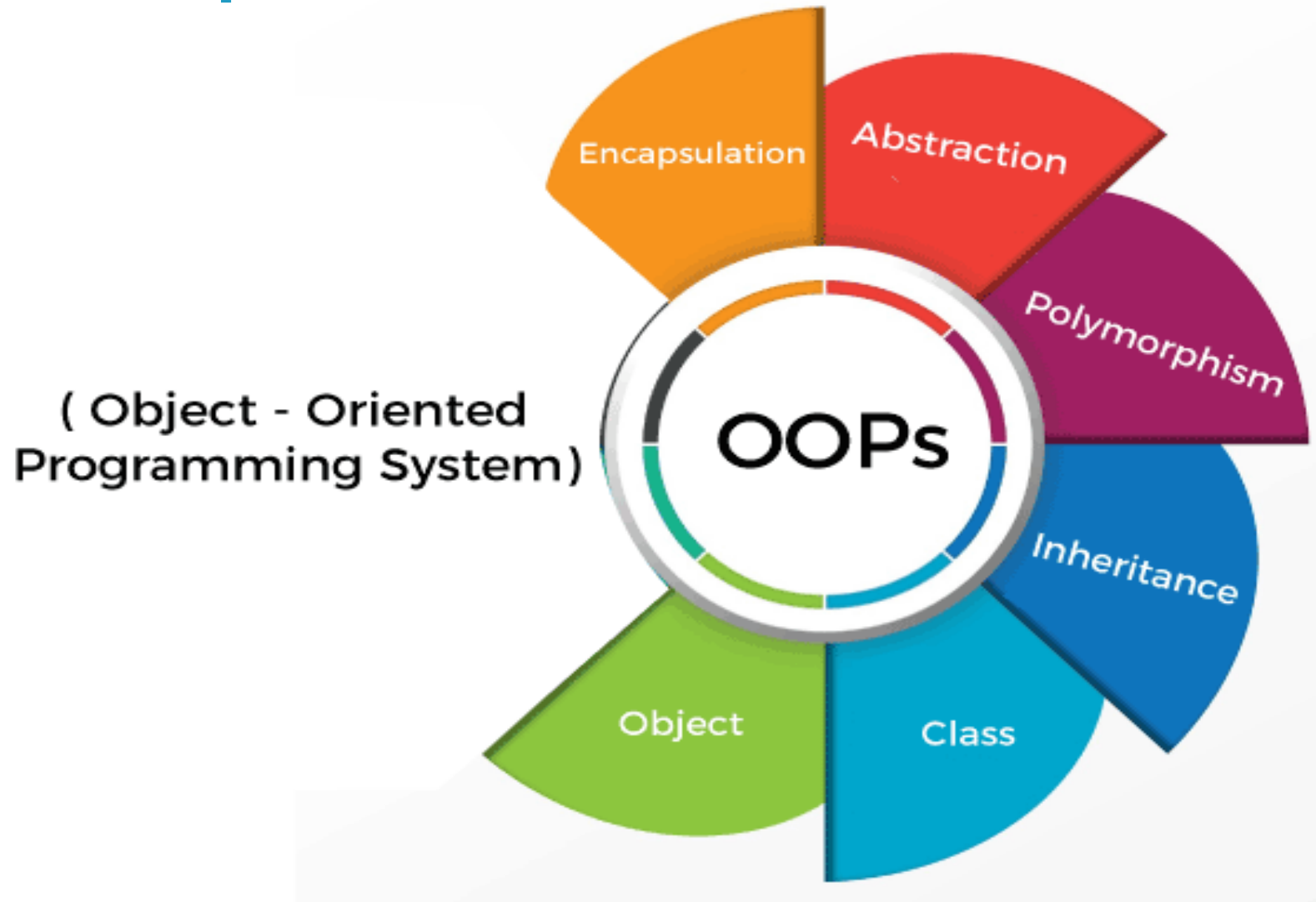
facturacion_telefono_funciones.Facturacion_telefono_funciones main for (int i = 1; i <= N; i++)

Output - facturacion_telefono_funciones (run)

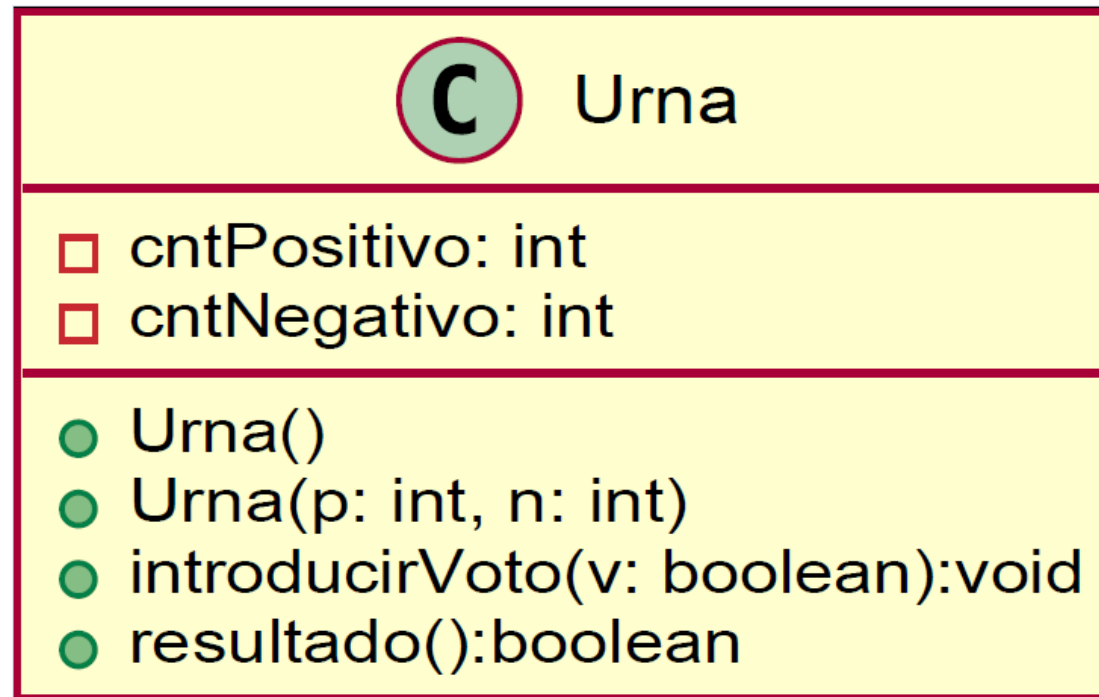
```
Impulsos mes:
100
Valor factura: 20000.0
Cedula:
2
Estrato(1,2,3,4,5):
4
Impulsos mes:
200
Valor factura: 45000.0
Valor total facturación: 65000.0
BUILD SUCCESSFUL (total time: 15 seconds)
```

INTRODUCCIÓN

PA - Conceptos P00 - OOP



PA - Conceptos P00 - Clase



PA - Conceptos P00 - Objeto

urna1 : Urna

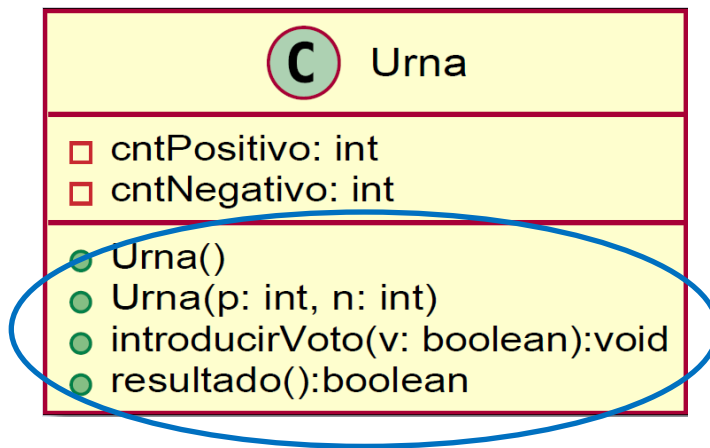
cntPositivo = 3
cntNegativo = 1

urna2 : Urna

cntPositivo = 2
cntNegativo = 2

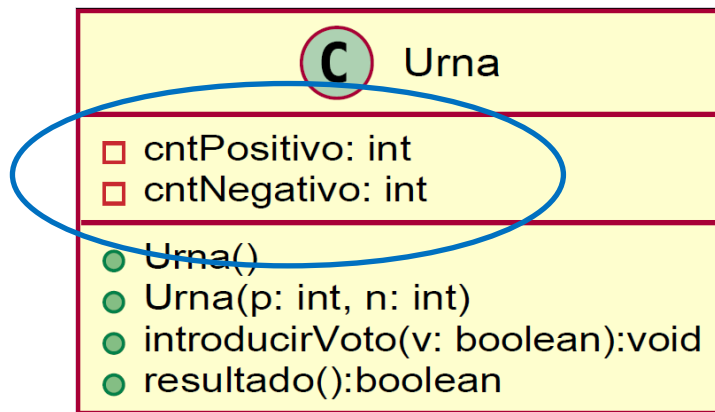
Un objeto se representa mediante una *caja* con dos compartimentos, conteniendo el primero el **nombre** del objeto y de la clase a la que pertenece, y el segundo los **valores** concretos de los atributos del objeto

PA - Conceptos P00 - Método



- La clase representa una abstracción de datos, y los métodos definen su comportamiento.
- Los métodos son algoritmos especiales definidos por la Clase, y se aplican sobre los objetos.
- Manipulan el estado interno del objeto sobre el que se aplican.
- También se les denomina métodos de instancia, o métodos del objeto.

PA - Conceptos P00 - Atributos



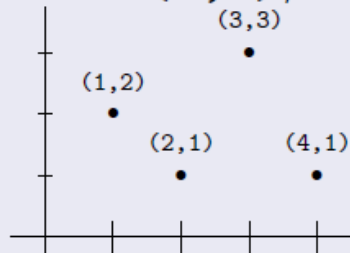
- Almacenan los valores del estado interno del objeto.
- Cada objeto tiene su propio estado interno asociado, independiente de los otros objetos.
- Los atributos están protegidos. Sólo se permite su acceso y manipulación a través de los métodos.
- También se les denomina variables de instancia, o variables del objeto.

PA - Conceptos P00 - Ejemplo

Abstracción: Punto del plano Cartesiano

Un *punto* representa una determinada posición en el plano Cartesiano.

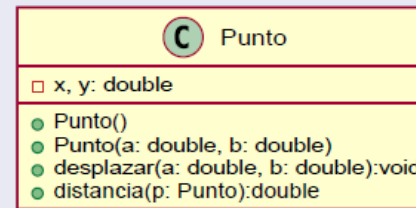
- Comportamiento de un *punto*:
 - Especificar el valor de sus coordenadas X e Y .
 - Consultar el valor de sus coordenadas X e Y .
 - Calcular la distancia que lo separa de otro objeto *punto*.
 - Desplazar según una distancia especificada en ambos ejes.
- Estado interno del *punto*:
 - El valor de la coordenada X (abscisa).
 - El valor de la coordenada Y (ordenada).
- Podemos crear múltiples objetos de la *Clase Punto*:
 - $\text{Punto}(1,2)$, $\text{Punto}(2,1)$, $\text{Punto}(3,3)$, $\text{Punto}(4,1)$



PA - Conceptos P00 – Representación Gráfica

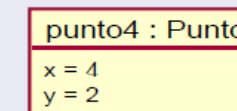
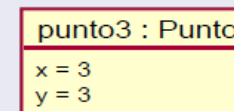
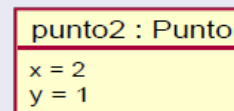
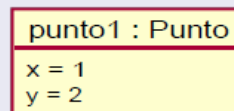
Representación Gráfica UML de Clases

- Una clase se representa mediante una *caja* con tres compartimentos, conteniendo cada uno de ellos el nombre, los atributos y los métodos de la clase.



Representación Gráfica UML de Objetos

- Un objeto se representa mediante una *caja* con dos compartimentos, conteniendo el primero el **nombre** del objeto y de la clase a la que pertenece, y el segundo los **valores** de los atributos del objeto.

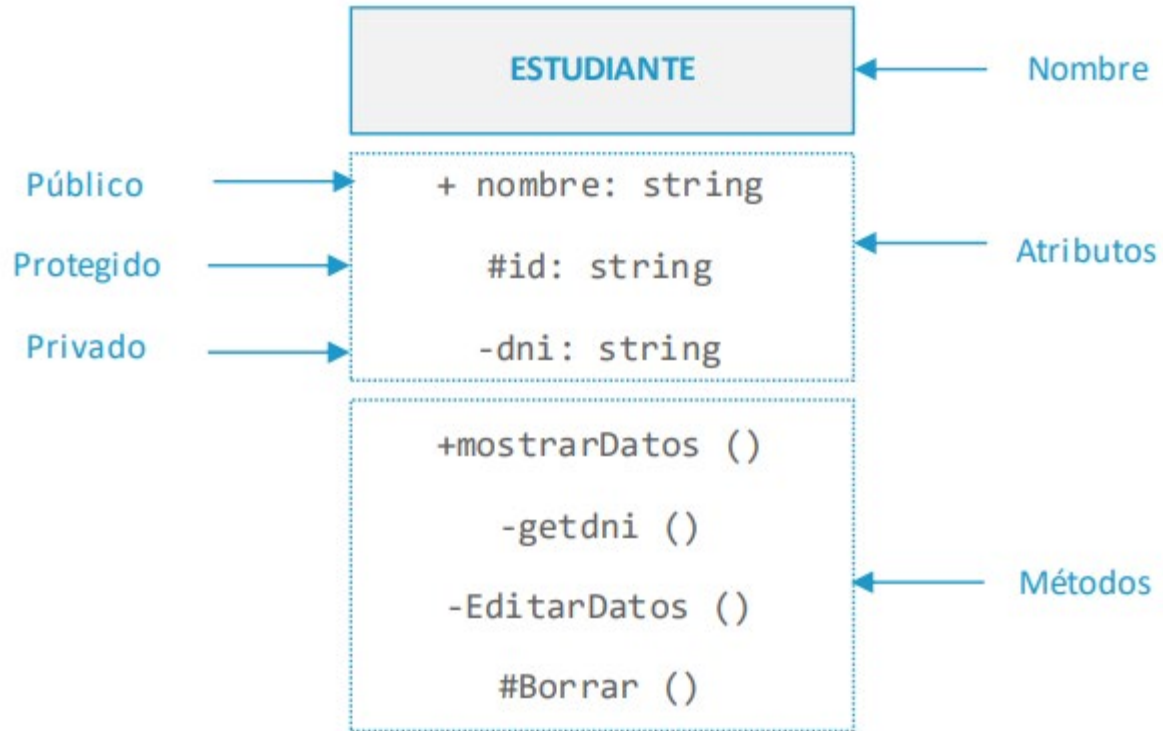


PA - Introducción a UML



El Lenguaje de Modelado Unificado (UML, Unified Modeling Language) es un lenguaje gráfico para visualizar, especificar y documentar cada una de las partes que comprende el desarrollo de software. UML entrega una forma de modelar cosas conceptuales como lo son procesos de negocio y funciones de sistema, además de cosas concretas como lo son escribir clases en un lenguaje determinado, esquemas de base de datos y componentes de software reutilizables.

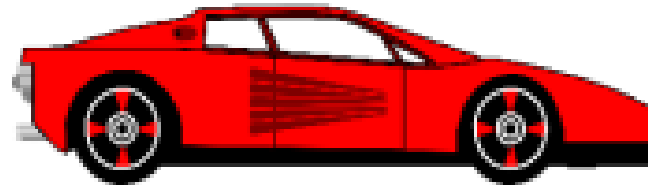
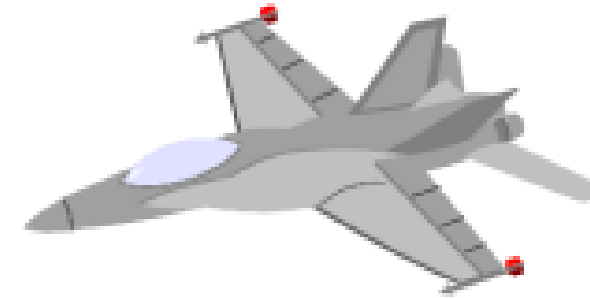
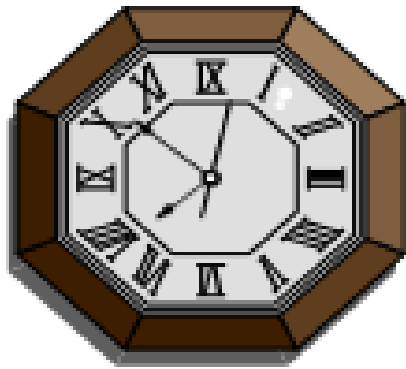
PA - Introducción a UML - Clase



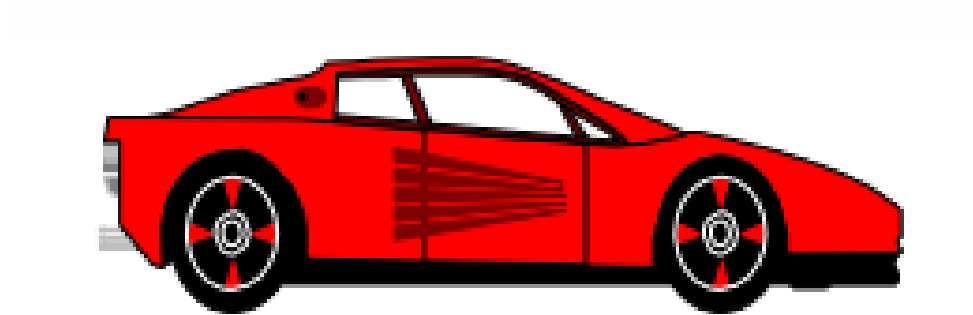
Access Levels

Modifier	Class	Package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N
no modifier	Y	Y	N	N
private	Y	N	N	N

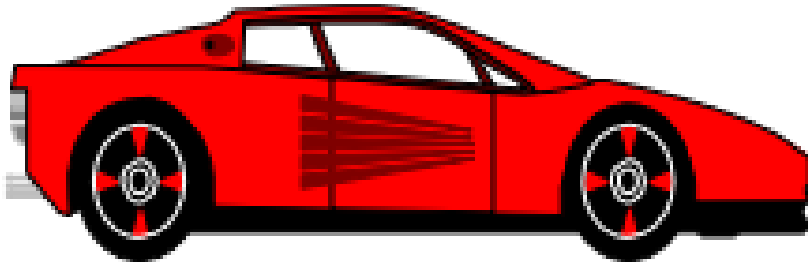
PA – Conceptos P00 - Objeto



PA – Conceptos P00 - Objeto



PA – Conceptos POO - Objeto



Es una forma de agrupar un conjunto de datos (**estado**) y de funcionalidad (**comportamiento**) en un mismo bloque de código que luego puede ser referenciado desde otras partes de un programa

PA – Conceptos P00 – Objeto - Ejemplo

```
public class Coche {
```

```
    private String color;  
    private int velocidad;  
    private float tamaño;
```

} Estado

```
    public Coche (String color, int velocidad, float tamaño){  
        this.color = color;  
        this.velocidad = velocidad;  
        this.tamaño = tamaño;  
    }
```

Constructor

```
    public void avanzar(){}  
    public void parar(){}  
    public void girarIzquierda(){}  
    public void girarDerecha(){}  
}
```

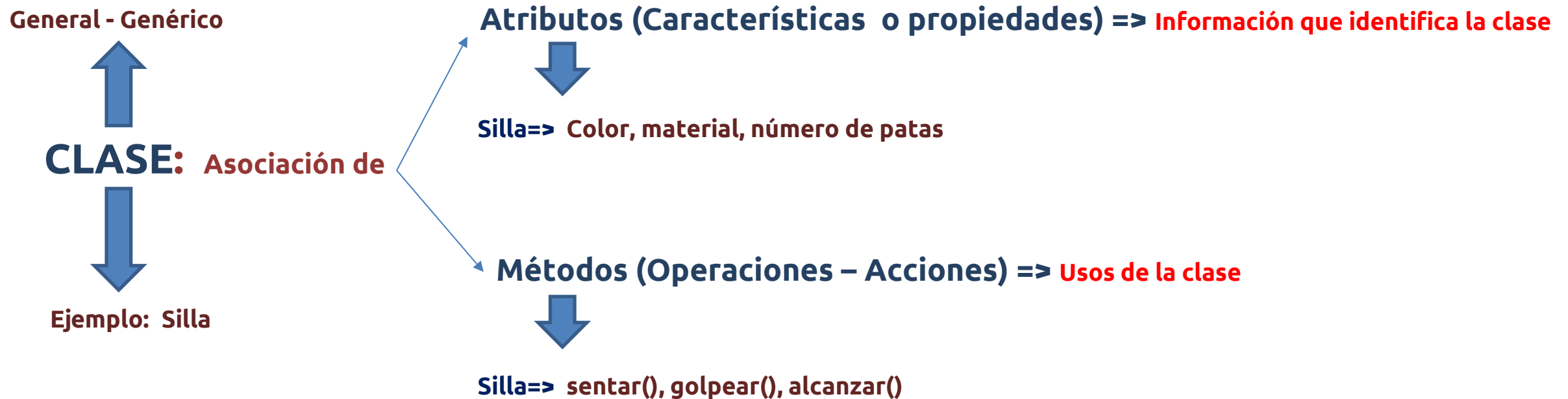
} Comportamiento

```
public static void main (String[] args){  
    Coche miCoche = new Coche ("verde", 80, 3.2f);  
    Coche tuCoche = new Coche ("rojo", 120, 4.1f);  
    Coche suCoche = new Coche ("amarillo", 100, 3.4f);  
}
```



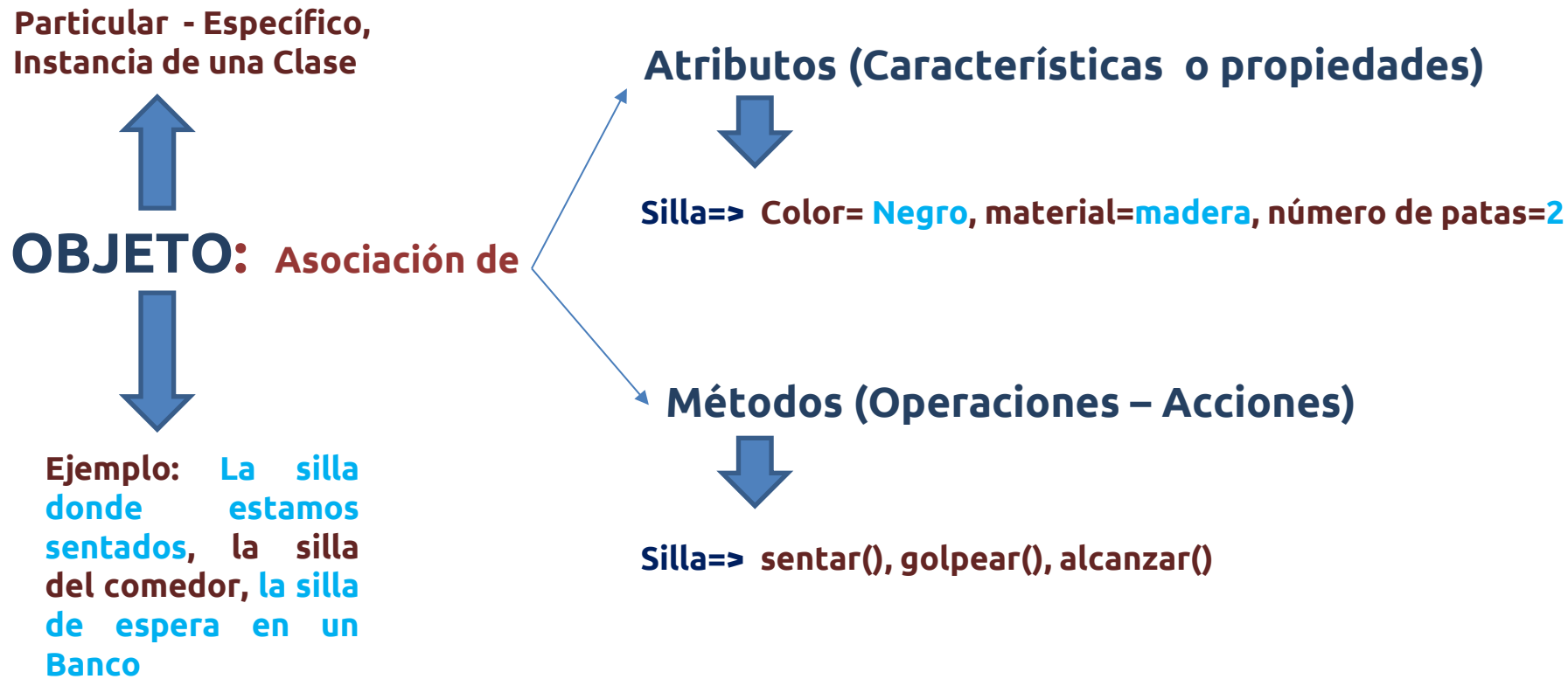
PA – Conceptos P00

Definición de Clase – Objeto (Atributos – Métodos)



PA – Conceptos P00

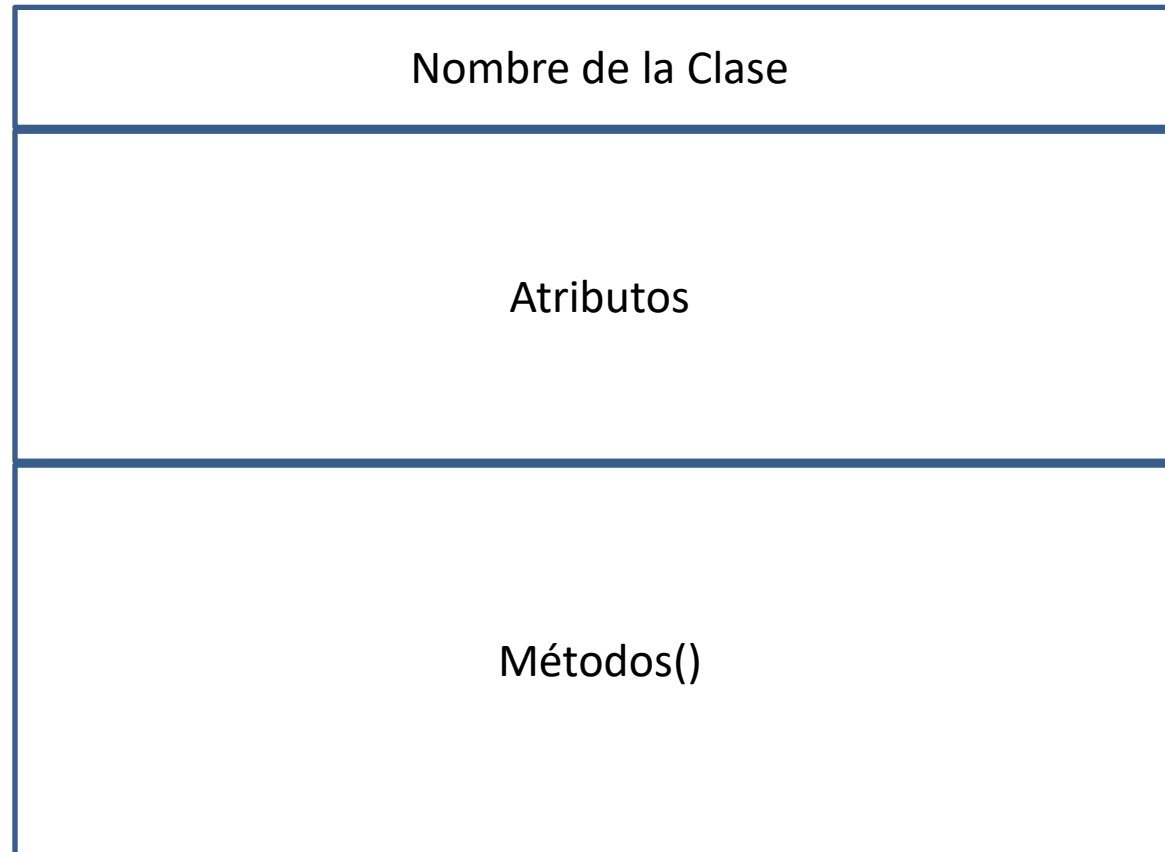
Definición de Clase – Objeto (Atributos – Métodos)



PA – Conceptos P00



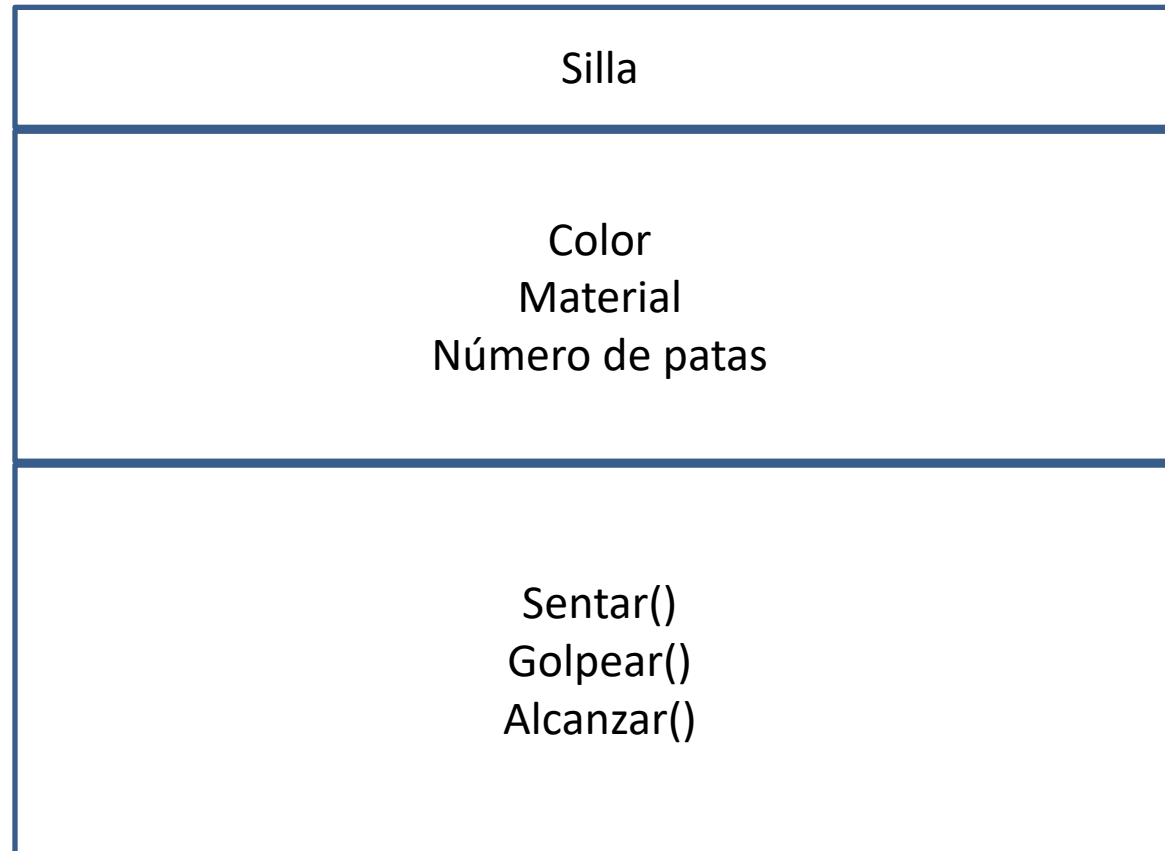
Representación de la Clase



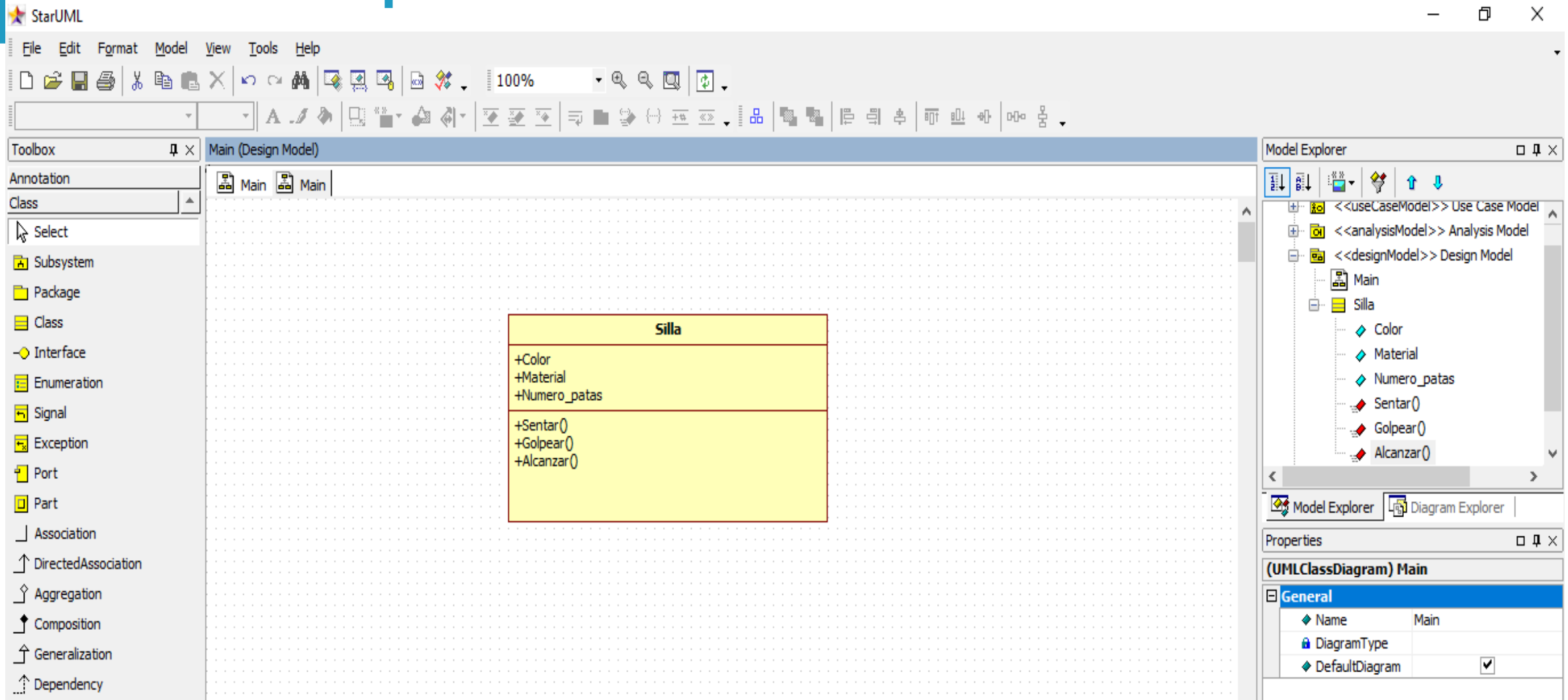
PA – Conceptos P00



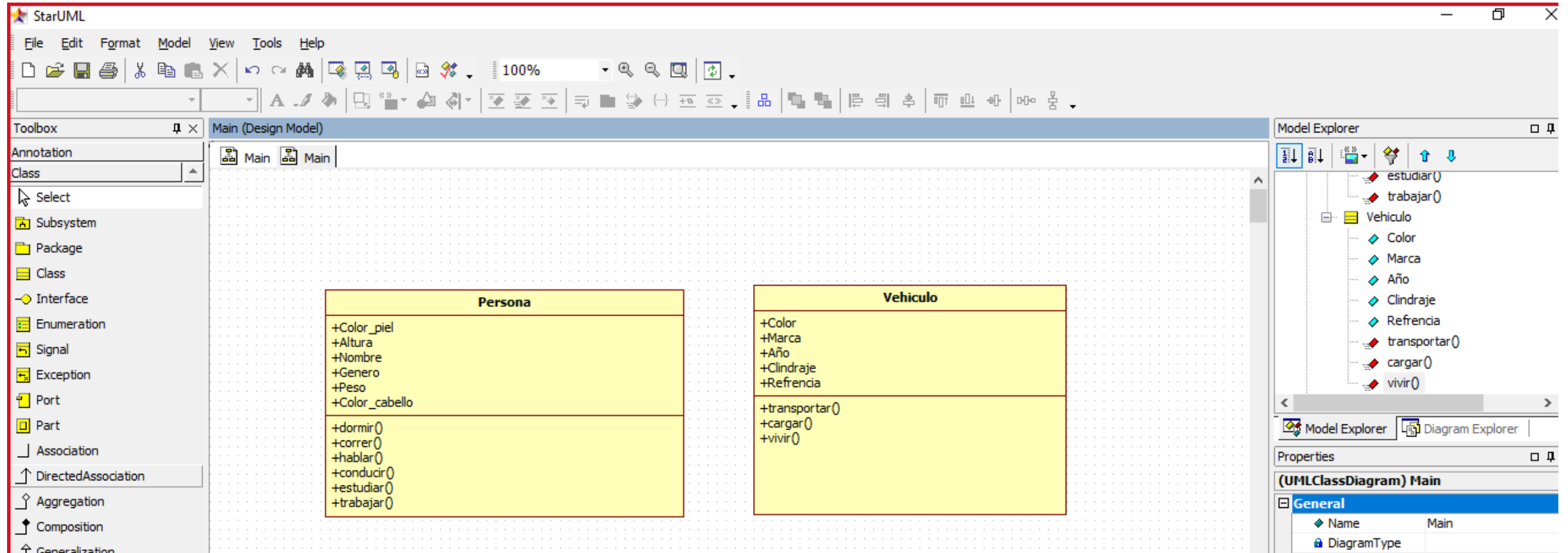
Representación de la Clase (Ejemplo de la Silla)



PA – Conceptos POO



PA – Conceptos P00



PA – Conceptos POO - Ejemplo

Dada la siguiente información sobre un **vendedor** de una empresa, del cual se conoce:

- **Documento de identidad**
- **Tipo Vendedor**(1=Puerta a Puerta, 2=Telemercadeo)
- **Valor ventas del mes**

Se pide calcular el valor a pagar por concepto de comisión al vendedor, de acuerdo con la siguiente indicación:

Para el vendedor de tipo 1(Puerta a Puerta) se le paga por concepto de comisión el 25% del valor de las ventas del mes. Cuando el vendedor es tipo 2(Telemercadeo) se le paga por concepto de comisión el 20% del valor de las ventas del mes.

Realizar el programa en Java que resuelva la situación problema presentada, utilizando el concepto de Clases y Objetos (POO).

PA – Conceptos P00 - Ejemplo



Constructor de una Clase

Un **constructor** es un elemento de una **clase** cuyo identificador coincide con el de la **clase** correspondiente y que tiene por objetivo obligar a y controlar cómo se **inicializa una instancia** de una determinada **clase**, ya que el lenguaje **Java** no permite que las variables miembro de una nueva instancia queden sin inicializar.

Un **constructor** es un método perteneciente a la clase que posee unas **características** especiales: Se llama igual que la clase. No devuelve nada, ni siquiera void. Pueden existir varios, pero siguiendo las reglas de la sobrecarga de funciones.

PA – Conceptos P00 - Ejemplo

Identificación de la Clase

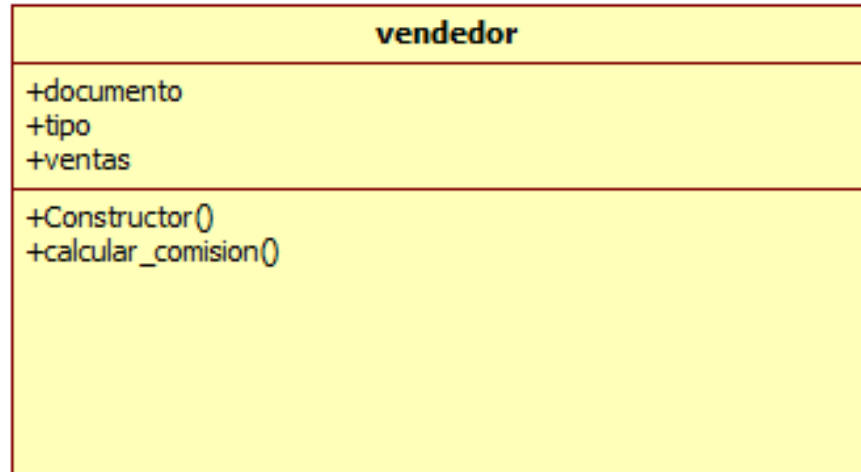
Nombre Clase

Atributos

Métodos

PA – Conceptos P00 - Ejemplo

Clase en StarUML



PA – Conceptos P00 - Ejemplo

Método

Parámetros de entrada

Tipo, ventas



Calcular_comisión():
Condicional (tipo de vendedor)



Parámetros de salida

comision

FUNCIÓN retorna o regresa un solo valor

PA – Conceptos P00 - Ejemplo

```
public class vendedor {  
    //Atributos  
    long documento;  
    int tipo;  
    double ventas;  
    //Métodos  
    public vendedor(long doc,int tipo,double ven){  
        this.documento=doc;  
        this.tipo=tipo;  
        this.ventas=ven;  
    }  
    public double calcular_comision(){  
        double comision;  
        if (this.tipo==1){  
            comision=this.ventas*0.25;  
        }  
        else{  
            comision=this.ventas*0.20;  
        }  
        return comision;  
    }  
}
```

PA – Conceptos P00 - Ejemplo

```
package comisiones_poo_g21;
```

```
/**  
 *  
 * @author SERGIO  
 */
```

```
import java.util.Scanner;
```

```
public class Comisiones_poo_G21 {
```

```
/**  
 * @param args the command line arguments  
 */
```

```
public static void main(String[] args) {  
    // Definición de la consola  
    Scanner consola=new Scanner(System.in);  
    //Definición de variables  
    long documento;  
    int tipo;  
    double ventas,comision;  
    //Definición de la variable para manejar el objeto  
    vendedor objeto_vendedor;
```

```
//Entrada de Datos
```

```
System.out.println("Documento: ");
```

```
documento=consola.nextLong();
```

```
System.out.println("Tipo(1=Puerta a puerta, 2=Telemercadeo): ");
```

```
tipo=consola.nextInt();
```

```
System.out.println("Ventas del mes: ");
```

```
ventas=consola.nextDouble();
```

```
//Creación del objeto
```

```
objeto_vendedor=new vendedor(documento,tipo,ventas);
```

```
comision=objeto_vendedor.calcular_comision();
```

```
System.out.println("Comisión: "+comision);
```


PA – Conceptos P00 – Objeto - Ejercicio

Implementa la clase **Bicicleta**, que tiene tres atributos, **velocidadActual**, **platoActual** y **piñonActual**, de tipo entero y cuatro métodos **acelerar()**, **frenar()**, **cambiarPlato(int plato)**, y **cambiarPiñon(int piñon)**, donde el primero dobla la velocidad actual, el segundo reduce a la mitad la velocidad actual, y el tercero y cuarto ajustan el plato y el piñón actual respectivamente según los parámetros recibidos. La clase debe tener además un constructor que inicialice todos los atributos.

Crea dos objetos de la clase bicicleta: **miBicicleta** y **tuBicicleta**

PA – Conceptos POO – Encapsulamiento

- **Encapsulación:** describe la vinculación de un comportamiento y un estado a un objeto en particular.
- **Ocultación de información:** Permite definir qué partes del objeto son visibles (el interfaz público) que partes son ocultas (privadas)

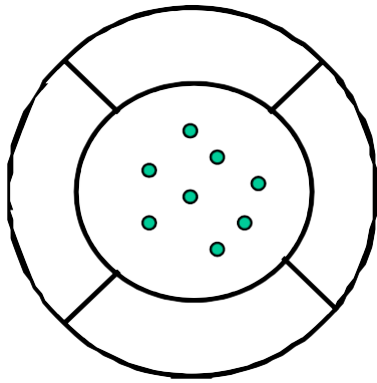


ventajas

- La llave de contacto es un interfaz público del mecanismo de arranque de un coche
- La implementación de cómo arranca realmente es privada y sobre ella sólo puede actuar la llave de contacto

El objeto puede cambiar y su interfaz pública ser compatible con el original: esto facilita reutilización de código

PA – Conceptos P00 – Encapsulamiento



MIEMBROS DE UNA CLASE

Los objetos encapsulan atributos permitiendo acceso a ellos únicamente a través de los métodos

- ▶ **Atributos (Variables):** Contenedores de valores
- ▶ **Métodos:** Contenedores de funciones

Un objeto tiene

- ▶ **Estado:** representado por el contenido de sus atributos
- ▶ **Comportamiento:** definido por sus métodos

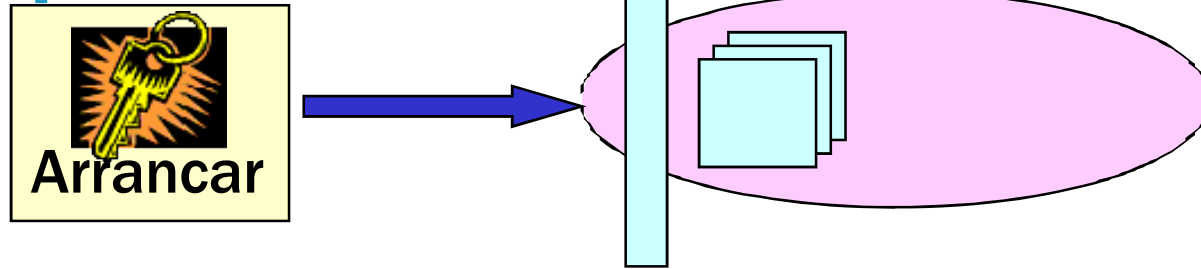
PA – Conceptos P00 – Encapsulamiento

- Miembros públicos
- los miembros públicos describen **qué** pueden hacer los objetos de esa clase
 - Qué pueden hacer
- los objetos (métodos)
 - Qué son los objetos (su abstracción)
- Miembros privados
- describen la implementación de
- **cómo** lo hacen.
 - Ejemplo: el objeto contacto interacciona con el circuito eléctrico del vehículo, este con el motor, etc.
 - *En sistemas orientados a objetos puros todo el estado es privado y sólo se puede cambiar a través del interfaz público.*
 - Ej: El método público frenar puede cambiar el valor del atributo privado velocidad.

PA – Conceptos P00 – Interacción

- El ***modelado de objetos*** modela:
 - Los objetos y
 - Sus interrelaciones
- Para realizar su tarea el objeto puede ***delegar*** trabajos en otro que puede ser parte de él mismo o de cualquier otro objeto del sistema.
- Los objetos interaccionan entre sí enviándose ***mensajes***

PA – Conceptos P00 – Paso de Mensajes



- Un objeto envía un **mensaje** a otro
 - Esto lo hace mediante una **llamada** a sus atributos o métodos
- Los mensajes son tratados por la **interfaz pública** del objeto que los recibe
 - Eso quiere decir que sólo podemos hacer llamadas a aquellos atributos o métodos de otro objeto que sean **públicos** o **accesibles** desde el objeto que hace la llamada
- El objeto receptor reaccionará
 - **Cambiando su estado**: es decir modificando sus atributos
 - **Enviando otros mensajes** : es decir llamando a otros atributos o métodos del mismo objeto (públicos o privados) o de otros objetos (públicos o accesibles desde ese objeto)

PA – Conceptos P00 – Paso de Mensajes - EJ

Clase Coche

```
public class Coche {  
  
    private String color;  
    private int velocidad;  
    private float tamaño;  
    private Rueda[] ruedas;  
    private Motor motor;  
  
    public Coche (String color, int velocidad,  
                  float tamaño, Rueda[] ruedas,  
                  Motor motor){  
        this.color = color;  
        this.velocidad = velocidad;  
        this.tamaño = tamaño;  
        this.ruedas = ruedas;  
        this.motor = motor;  
    }  
  
    public void avanzar(){  
        motor.inyectarCarburante();  
        for (int i=0; i < ruedas.length; i++){  
            ruedas[i].girar();  
        }  
    }  
  
    public static void main (String[] args){  
  
        Rueda[] ruedas = {new Rueda(20,"Dunlop"),  
                           new Rueda(20,"Dunlop"),  
                           new Rueda(22, "Dunlop"),  
                           new Rueda(22, "Dunlop")};  
        Coche miCoche = new Coche ("verde", 80,3.2f,  
                                     ruedas, new Motor("Diesel",100));  
    }  
}
```

Clase Motor

```
public class Motor {  
  
    private String tipo;  
    private int caballos;  
  
    public Motor(String tipo, int caballos){  
        this.tipo = tipo;  
        this.caballos = caballos;  
    }  
  
    public void inyectarCarburante(){...}  
}
```

Clase Rueda

```
public class Rueda {  
  
    private double diametro;  
    private String fabricante;  
  
    public Rueda (double diametro, String fabricante){  
        this.diametro = diametro;  
        this.fabricante = fabricante;  
    }  
  
    public void girar(){...}  
}
```

Recordar

RA 1

RA 2

RA 3

Actividad 1

Actividad 2

Test



「muchas gracias」